



UNIVERSITÄT
LEIPZIG

Fakultät

Institut

Fachgebiet

Erstbetreuer

Zweitbetreuer

Thema

Titel

Bachelorarbeit zur Erlangung des akademischen Grades

Bachelor of Science – *Studiengang*

vorgelegt von: *Autorenname*

Matrikelnummer: *Matrikelnummer*

E-Mail-Adresse: *E-Mail*

Telefonnummer: *Telefonnummer*

Anschrift: *Straße*
Plz und Ort

Leipzig, den Datum

Zusammenfassung

Dein Abstract

Schlüsselwörter:

Deine Schlüsselwörter

Inhaltsverzeichnis

Abbildungsverzeichnis

Tabellenverzeichnis

Formelverzeichnis

Abkürzungsverzeichnis

ML Machine Learning

1 Einleitung

1.1 Motivation und Problemstellung

Die Landwirtschaft steht vor einer Reihe gleichzeitiger Herausforderungen: Der Klimawandel führt zu erhöhtem Schädlingsdruck, veränderten Niederschlagsmustern und sinkenden Erträgen (Calvin u. a. 2023; Hultgren u. a. 2025; *Climate Change Trends and Impacts on California Agriculture: A Detailed Review* | MDPI 2026), während eine wachsende Weltbevölkerung und zunehmender Fachkräftemangel den Produktionsdruck weiter verschärfen (Food and Agriculture Organization of the United Nations 2017; *World Population Prospects* 2026; Duckett u. a. 2018). Als Teilgebiet der digitalen Landwirtschaft (Basso und Antle 2020) bieten unbemannte Luftfahrzeuge (UAVs) vielversprechende Lösungsansätze: Sie ermöglichen unter anderem die gezielte Ausbringung von Pflanzenschutzmitteln sowie großflächiges Crop-Monitoring mittels Remote Sensing (Mashori u. a. 2026; Radoglou-Grammatikis u. a. 2020) und können so dazu beitragen, die Landwirtschaft effizienter und resilienter zu gestalten (Unde u. a. 2025; Guebsi u. a. 2024).

Trotz umfangreicher wissenschaftlicher Arbeit zu Drohnen gibt es noch viele Themengebiete mit Entwicklungspotential, wie beispielsweise autonomes Landen (Amendola u. a. 2024). Für die meisten Flugmissionen ist unfallfreies Landen essentiell und es ist entscheidend Landetechnologien weiter auszubauen, um das gesamte Potential autonomer Drohnen ausnutzen zu können (Amendola u. a. 2024). Um unfallfreies Landen sicherzustellen, sollten die Flugmanöver in der Landephase präzise und verlässlich sein. Aufgrund der durchaus hohen Anforderungen und Vielzahl verschiedenster Landeszenarien ist autonomes Landen von Drohnen Thema vieler Forschungsarbeiten **einfach mehrere Forschungsarbeiten auflisten weil die meisten Review Paper schwachsinn sind.**

Arbeiten kurz beschreiben, spezielle szenarien, vereinfacht, unterschiedliche Systeme

Landesysteme lassen sich in 3 Kategorien einteilen: GNSS-basierte Systeme, Sensordaten-Fusionssysteme und Bild-basierte Systeme.

GNSS-basierte Systeme verfügen über Positions- und Geschwindigkeitsdaten durch Satelliten. Solche Systeme verfügen über hohe Genauigkeit, bis auf zentimetergenaue Präzision.

Sensordaten-Fusionssysteme kombinieren Daten von mehreren Sensoren wie GPS, IMU oder LiDAR zur Positionbestimmung und Navigation. Verschiedene Studien zeigen, dass au-

tonomes Landen oder Navigieren in verschiedenen komplexen Szenarien mit diesem Ansatz möglich ist **QUELLEN**.

Bild-basierte Ansätze nutzen Bilddaten zur Positionsbestimmung und Umgebungsanalyse. Sie sind im Vergleich "leichtgewichtig" da nur RGB-Kameras als Sensoren benötigt werden. Die Integration von Kamerasensoren ermöglicht es den Landesystemen sich dynamisch an ihre Umgebung anzupassen.

In vergangenen Jahren hat sich Reinforcement Learning als ein leistungsstarkes Paradigma zur Lösung verschiedener Drohnenaufgaben entwickelt (Kaufmann u. a. 2023; Hodge u. a. 2021). Auch Arbeiten zum Landen von Drohnen haben vermehrt Reinforcement Learning zur Problemlösung eingesetzt, besonders bei komplexen Aufgaben (Amendola u. a. 2024). Reinforcement Learning als lernbasierter Ansatz hat das Potential, die Limitationen klassischer Kontroll-Algorithmen zu überwinden und es der Drohne zu ermöglichen, sich verschiedenen Szenarios anzupassen. In Kombination mit Convolutional Neural Networks (CNNs) können RL-Agenten aufgabenrelevante Merkmale direkt aus Bilddaten extrahieren, ohne dass diese manuell definiert werden müssen (Mnih, Kavukcuoglu u. a. 2015). Insbesondere die bildbasierte Navigation hat durch Fortschritte in der CNN-Architekturforschung deutlich profitiert, da tiefere und effizientere Netzwerke eine robustere Merkmalsextraktion aus verrauschten Sensordaten ermöglichen (Loquercio u. a. 2021).

Trotz der Fortschritte in einzelnen Ansätzen bringt das autonome Landen in natürlichen Umgebungen besondere Herausforderungen mit sich, die von keinem einzelnen System vollständig adressiert werden. GNSS-basierte Systeme liefern zwar globale Positionsdaten, sind jedoch nicht in der Lage, auf lokale Gegebenheiten wie unebenes Terrain, Vegetation oder unvorhergesehene Hindernisse zu reagieren **QUELLEN?**. Rein visuelle Ansätze können solche lokalen Informationen erfassen, sind aber anfällig gegenüber variierenden Lichtverhältnissen, Verdeckungen durch Pflanzen **QUELLEN?**. Zusätzlich ist es nicht möglich, bild-basierten Systemen ein Missionsziel in Form einer Koordinate zu geben, eine klare Limitation. Während zusätzliche Sensoren es der Drohne ermöglichen, mehr Informationen aus der lokalen Umgebung zu erfassen, würden sie auch Gewicht, Energieverbrauch und Kosten der Drohne erhöhen. Sensoren wie GPS und Kamera sind auf nahezu allen kommerziellen Drohnenmodellen verfügbar. Eine kleine Anzahl an Sensoren sorgt für breitere Anwendbarkeit und längere Flugzeit aufgrund von geringerem Gewicht und Energieverbrauch. Die vorliegende Arbeit präsentiert eine Reinforcement-Learning Pipeline welche GPS-basierte Zielnavigation mit bildbasierter Hinderniserkennung verbindet.

1.2 Zielstellung

Ziel der Arbeit ist die Entwicklung einer Reinforcement-Learning-Pipeline, die eine autonome Drohne befähigt, mithilfe lokaler Tiefensensorik in einer hindernisreichen Umgebung sicher zu landen. Das System kombiniert einen propriozeptiven Zustandsvektor mit tiefenbildbasierter Umgebungswahrnehmung, um Hindernisse während des Landevorgangs zu erkennen und ihnen auszuweichen. Die Leistungsfähigkeit des trainierten Agenten wird anhand einer abstrakten Trainingsumgebung evaluiert und anschließend im Zero-Shot-Transfer auf eine Umgebung mit realistischer Vegetationsstruktur getestet.

Dadurch ergeben sich folgende Fragestellungen:

Hauptfragestellung

Inwiefern kann ein Reinforcement-Learning-basierter Ansatz eine autonome Drohne befähigen, mithilfe lokaler Tiefensensorik in hindernisreichen Umgebungen sicher zu landen?

Nebenfragestellungen

1. Wie leistungsfähig ist der trainierte Agent in der Trainingsumgebung, und welche Faktoren begrenzen die Erfolgsrate?
2. In welchem Maß lässt sich die erlernte Navigationsleistung auf eine geometrisch verschiedenartige Umgebung mit realistischer Vegetationsstruktur übertragen?
3. Welche Leistungslücke besteht zwischen einem lernbasierten Agenten mit lokaler Sensorik und einem Pfadplaner mit vollständiger Umgebungsinformation?
4. Welchen Beitrag leisten die einzelnen Beobachtungskomponenten, insbesondere die visuelle Tiefenwahrnehmung und der propriozeptive Zustandsvektor, zur Navigationsleistung des Agenten?

1.3 Aufbau der Arbeit

2 Fachliche Grundlagen

2.1 Reinforcement Learning

Die folgenden Grundlagen basieren auf Sutton und Barto 2020.

2.1.1 Allgemein

Reinforcement Learning bedeutet, aus Interaktionen zu lernen, um ein Ziel zu erfüllen. Dafür muss der Lerner erfahren, welche Handlungen in welchen Situationen auszuführen sind, um eine bestimmte Belohnung zu maximieren. Dabei muss er selbstständig herausfinden, welche Aktionen den höchsten Ertrag generieren, wobei nicht nur unmittelbare, sondern auch zukünftige Belohnungen zu berücksichtigen sind.

Die wesentlichen Elemente eines Reinforcement-Learning-Systems sind: Agent, Umgebung, Strategie, Belohnung, Wert-Funktion und Modell der Umgebung. Im Folgenden werden die Elemente des Reinforcement Learning näher erläutert.

2.1.2 Markov Decision Process

Ein *Markov Decision Process* (MDP) ist ein Modell, mit dem Reinforcement Learning Probleme präzise formalisiert werden können. Der Lerner und Entscheider wird als *Agent* bezeichnet; alles außerhalb, womit der Agent interagiert und was er nicht willkürlich verändern kann, bildet die *Umgebung*.

Die Interaktion zwischen Agent und Umgebung erfolgt in diskreten Zeitschritten $t = 0, 1, 2, \dots$. In jedem Schritt beobachtet der Agent einen *Zustand* $S_t \in \mathcal{S}$, wählt auf dessen Grundlage eine *Aktion* $A_t \in \mathcal{A}$ und erhält einen Zeitschritt später eine numerische *Belohnung* $R_{t+1} \in \mathcal{R}$ sowie einen neuen Zustand S_{t+1} . Aus dieser fortlaufenden Interaktion entsteht die Sequenz

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \dots \quad (2.1)$$

Die grafische Darstellung dieses Zyklus findet sich in Abbildung ??.

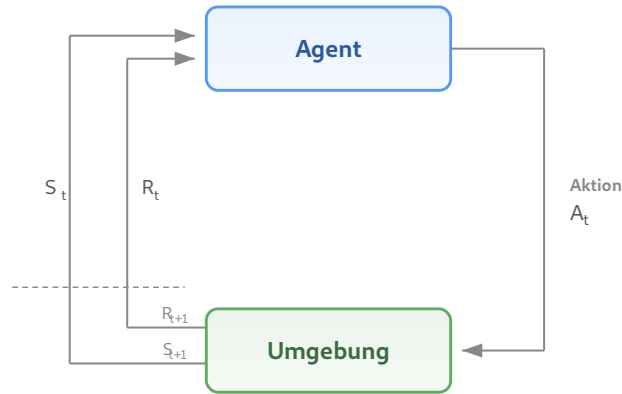


Abbildung 2.1.: Interaktion zwischen Agent und Umgebung in einem MDP (nach Sutton und Barto 2020, S. 48).

Die Übergangsdynamik der Umgebung wird durch die Funktion

$$p(s', r \mid s, a) = \Pr\{S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a\} \quad (2.2)$$

vollständig charakterisiert, welche die Wahrscheinlichkeit angibt, ausgehend vom Zustand s bei gewählter Aktion a in den Folgezustand s' überzugehen und dabei die Belohnung r zu erhalten. Wesentlich ist dabei die *Markov-Eigenschaft*: Folgezustand und Belohnung hängen ausschließlich vom gegenwärtigen Zustand-Aktions-Paar (s, a) ab.

Das Ziel des Agenten besteht darin, durch Interaktion mit der Umgebung eine Strategie $\pi(a \mid s)$ zu identifizieren, welche die über die Zeit kumulierte Belohnung maximiert. Eine solche Strategie wird als *optimale Strategie* bezeichnet und mit π_* gekennzeichnet.

2.2 Lösungen von MDPs

Welche Verfahren zur Bestimmung einer optimalen Strategie geeignet sind, hängt maßgeblich von der Formulierung des Reinforcement-Learning-Problems ab. Die klassische Theorie liefert exakte Lösungen ausschließlich für endliche MDPs mit vollständig bekanntem Übergangsmodell; ihre Grundideen lassen sich jedoch auf komplexere Problemklassen verallgemeinern. Dieser Abschnitt behandelt die Lösung von MDPs. Eingeführt werden zunächst tabellarische Verfahren, anschließend Funktionsapproximation und schließlich Policy-Gradient-Methoden, die in das Verfahren der *Proximal Policy Optimization* (PPO) münden.

2.2.1 Dynamische Programmierung

Das Ziel des Agenten besteht in der Maximierung des erwarteten Ertrags $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$ mit Diskontierungsfaktor $\gamma \in [0, 1)$. Die Güte einer Strategie π wird durch zwei Funktionen

charakterisiert: die Zustandswertfunktion $v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$ und die Aktionswertfunktion $q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$, die den erwarteten Ertrag eines Zustands bzw. einer in diesem Zustand gewählten Aktion angeben.

Eine Strategie π wird als mindestens so gut wie eine Strategie π' bezeichnet, falls $v_\pi(s) \geq v_{\pi'}(s)$ für alle $s \in \mathcal{S}$ gilt. Es existiert stets (mindestens) eine Strategie, die in diesem Sinne allen übrigen überlegen ist; sie wird als optimale Strategie π_* bezeichnet.

Die zugehörige Zustandswertfunktion $v_* = v_{\pi_*}$ heißt optimale Zustandswertfunktion und erlaubt die unmittelbare Ableitung von π_* .

Sind das Übergangsmodell der Umgebung vollständig bekannt und der Zustandsraum $\mathcal{S}$ abzählbar, so ergibt sich v_* als Lösung der Bellman-Optimalitätsgleichung:

$$v_*(s) = \max_{a \in \mathcal{A}} \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_*(s')]. \quad (2.3)$$

Die optimale Strategie lässt sich auf diesem Weg exakt durch Lösen des resultierenden nicht-linearen Gleichungssystems bestimmen.

In der Praxis sind beide Voraussetzungen jedoch selten gleichzeitig erfüllt: Das Übergangsmodell physikalischer Systeme ist in der Regel nicht vollständig bekannt, und selbst bei bekanntem Modell übersteigt der Zustandsraum realistischer Probleme typischerweise jene Dimensionen, in denen sich die Bellman-Optimalitätsgleichung mit vertretbarem Rechenaufwand lösen ließe.

2.2.2 Monte-Carlo- und Temporal-Difference-Verfahren

Die erste dieser Einschränkungen – das fehlende Modell – wird durch *sample-basierte* Verfahren wie Monte-Carlo- und Temporal-Difference-Methoden (TD) aufgehoben. Monte-Carlo-Methoden benötigen den vollständigen Episodenertrag G_t und können die Wertschätzung daher erst nach Abschluss einer Episode aktualisieren. TD-Verfahren hingegen nutzen *Bootstrapping* und korrigieren ihre Schätzung nach jedem einzelnen Zeitschritt anhand der geschätzten Bewertung des Folgezustands:

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]. \quad (2.4)$$

TD-Verfahren bilden die Grundlage der meisten modernen Reinforcement-Learning-Algorithmen: Sie können online angewandt werden, benötigen wenig Rechenaufwand und lassen sich mit Funktionsapproximation auf große Zustandsräume skalieren.

2.2.3 Funktionsapproximation

Die zweite Einschränkung – die Dimensionalität des Zustandsraums – kann durch *Funktionsapproximation* überwunden werden. Statt jeden Zustand mit einem eigenen Eintrag zu versehen, wird die Wertfunktion durch eine parametrisierte Funktion $\hat{v}(s, \mathbf{w})$ mit $\mathbf{w} \in \mathbb{R}^d$ und $d \ll |S|$ dargestellt und in Richtung v_π optimiert. Parametrisierte Funktionen erlauben Generalisierung: Erfahrungen, die über einen Zustand gemacht wurden, lassen sich auf benachbarte Zustände übertragen; gleichzeitig werden kontinuierliche Zustandsräume handhabbar. Als Parametrisierung eignen sich insbesondere neuronale Netze, da diese als universelle Funktionsapproximatoren in der Lage sind, beliebige stetige Funktionen auf kompakten Teilmengen beliebig genau anzunähern (Hornik u. a. 1989).

Während die Funktionsapproximation das Problem kontinuierlicher Zustandsräume löst, bleibt ein zweites Hindernis bei kontinuierlichen Aktionsräumen bestehen. Wertbasierte Verfahren wählen Aktionen gemäß $a = \arg \max_{a'} \hat{q}(s, a', \mathbf{w})$; dieses Maximum ist in kontinuierlichen Aktionsräumen jedoch im Allgemeinen nicht effizient berechenbar.

2.2.4 Policy-Gradient-Methoden

Eine Alternative ist, die Strategie nicht über eine Wert-Funktion abzuleiten, sondern direkt als parametrisierte Funktion $\pi(a \mid s, \theta)$ zu beschreiben und ihre Parameter θ durch Gradientenaufstieg zu optimieren. Das *Policy-Gradient-Theorem* drückt diesen Gradienten aus als

$$\nabla J(\theta) \propto \sum_s \mu(s) \sum_a q_\pi(s, a) \nabla \pi(a \mid s, \theta), \quad (2.5)$$

wobei μ die Zustandsbesuchsverteilung unter π bezeichnet. Der entscheidende Vorteil für diese Arbeit: Policy-Gradient-Methoden können auf natürliche Weise kontinuierliche Aktionsräume handhaben, wie sie in der Drohnensteuerung auftreten.

In der Praxis wird der Policy-Gradient-Ansatz häufig als *Actor-Critic-Architektur* umgesetzt: Ein *Actor*-Netz repräsentiert die Strategie $\pi(\cdot \mid \cdot, \theta)$, ein *Critic*-Netz schätzt eine Wertfunktion $\hat{v}(\cdot, \mathbf{w})$. Die Wertschätzung des Critics dient als Referenzniveau, das die Varianz der Gradientenschätzung reduziert, ohne sie zu verzerren.

2.2.5 Proximal Policy Optimization

Eine moderne und in der Praxis weit verbreitete Variante des Policy-Gradient-Ansatzes ist *Proximal Policy Optimization* (PPO) (Schulman, Wolski u. a. 2017), ein Actor-Critic-

Verfahren, das die Trainingsstabilität von Trust-Region-Methoden erreicht und dabei wesentlich einfacher zu implementieren ist.

Zu große Strategieänderungen pro Optimierungsschritt können Policy-Gradient-Verfahren destabilisieren, da die gesammelten Trajektorien für die veränderte Strategie nicht mehr repräsentativ sind (Schulman, Wolski u. a. 2017). Trust-Region-Methoden wie TRPO (Schulman, Levine u. a. 2015) begrenzen daher die KL-Divergenz zwischen aufeinanderfolgenden Strategien, erfordern dafür jedoch Approximationen zweiter Ordnung. PPO erzielt eine vergleichbare Stabilisierung allein durch eine geklippte Zielfunktion:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right], \quad (2.6)$$

wobei $r_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{alt}}}(a_t | s_t)$ das Wahrscheinlichkeitsverhältnis zwischen neuer und alter Strategie ist, $\hat{A}_t$ der Vorteilsschätzer und ε die Clipping-Grenze. Das Clipping bewirkt *pessimistische* Updates: Verbessert eine Strategieänderung das Objective, wird der Anreiz bei $r_t > 1 + \varepsilon$ gekappt; verschlechtert sie es, greift die Minimumbildung und die Korrektur bleibt uneingeschränkt (Abbildung ??). So werden zu große Schritte in Richtung guter Erfahrungen gebremst, während schlechte Aktionen voll korrigiert werden.

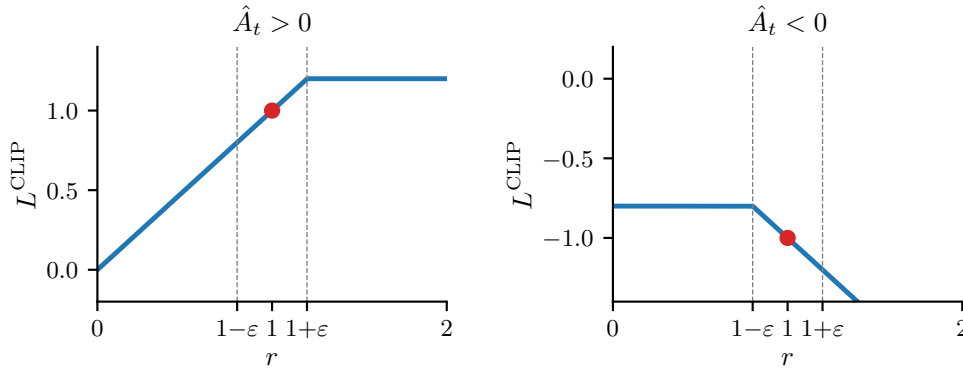


Abbildung 2.2.: L^{CLIP} als Funktion des Wahrscheinlichkeitsverhältnisses r . Links ($\hat{A}_t > 0$): der Anreiz wird bei $1 + \varepsilon$ gekappt. Rechts ($\hat{A}_t < 0$): die Korrektur bleibt uneingeschränkt (nach (Schulman, Wolski u. a. 2017), Fig. 1).

Das vollständige PPO-Objective kombiniert das Clipping mit zwei weiteren Termen:

$$L(\theta) = L^{\text{CLIP}}(\theta) - c_1 L^{VF}(\theta) + c_2 H[\pi_\theta](s_t), \quad (2.7)$$

wobei $L^{VF} = (V_\theta(s_t) - V_t^{\text{targ}})^2$ der quadratische Fehler der Wertfunktionsschätzung des Critics ist und $H[\pi_\theta]$ einen Entropie-Bonus bezeichnet, der vorzeitige Konvergenz der Aktionsverteilung verhindert. Ohne diesen Term kann die Standardabweichung gegen null konvergieren (*Std-Collapse*), was die Strategie irreversibel auf eine deterministische Aktion festlegt.

Für die Schätzung des Vorteils $\hat{A}_t$ wird *Generalized Advantage Estimation* (GAE) (Schulman, Moritz u. a. 2016) eingesetzt, das über den Parameter λ einen Kompromiss zwischen Bias und Varianz der Vorteilsschätzung steuert.

Schulman, Wolski u. a. (2017) evaluieren PPO auf zwei Benchmark-Suiten: In den MuJoCo-Continuous-Control-Benchmarks übertrifft PPO die Vergleichsverfahren TRPO (Schulman, Levine u. a. 2015) und A2C (Mnih, Badia u. a. 2016) auf nahezu allen Umgebungen. In der Arcade Learning Environment zeigt PPO ebenfalls kompetitive Ergebnisse und gewinnt 30 von 49 Spielen nach durchschnittlicher Episodenbelohnung über das gesamte Training (Schulman, Wolski u. a. 2017). Der Clipping-Mechanismus erzeugt dabei ein stabiles Konvergenzverhalten, das die Robustheit von Trust-Region-Algorithmen erreicht, ohne deren Implementierungskomplexität zu erfordern.

3 Stand der Forschung

3.1 Taxonomie der Ansätze

Bisherige Arbeiten zum autonomen Landen von Drohnen lassen sich in zwei grundlegende Kategorien einteilen: traditionelle und bildbasierte Methoden. Traditionelle Methoden stützen sich auf Signale externer Infrastruktur wie GNSS-Satelliten oder Funkbaken, um Position und Lage der Drohne zu bestimmen. Bildbasierte Methoden hingegen nutzen Kameradaten zur Wahrnehmung der Umgebung und lassen sich weiter unterteilen in markerbasierte Systeme, die definierte visuelle Referenzpunkte zur Lokalisierung verwenden, und markerfreie Systeme, die adäquate Landezonen eigenständig aus Bilddaten ableiten und somit keine zusätzliche Bodeninfrastruktur erfordern. Diese Einteilung orientiert sich an der Taxonomie von Arafat u. a. (2023).

Orthogonal zur Sensorik lassen sich Ansätze danach unterscheiden, welche Teilaufgaben des Guidance, Navigation and Control (GNC) (Kanellakis und Nikolakopoulos 2017) sie adressieren und wie diese zusammenwirken. Modulare Methoden zerlegen das Problem in getrennte Komponenten wie Obstacle Detection, Pose-Estimation oder Path-Planning, die als aufeinanderfolgende Stufen einer Pipeline gelöst werden (Kanellakis und Nikolakopoulos 2017; Wang u. a. 2024). End-to-End-Methoden hingegen bilden Sensorinput direkt auf Steuerkommandos ab, ohne solche expliziten Zwischenstufen (Wang u. a. 2024).

3.2 GNSS-basierte Methoden

GNSS ist die verbreitetste Methode zur Positionsbestimmung von Drohnen: Ein einzelnes GNSS-Modul liefert eine Genauigkeit von 5–10 m und bildet die Grundlage für Missionsplanung und -ausführung (Jarraya u. a. 2025). Dass autonomes Landen bei bekannter Relativposition grundsätzlich beherrschbar ist, zeigen Voos und Bou-Ammar (2010) mittels nichtlinearer Regelung auf stationären und beweglichen Zielen. Die zentrale Herausforderung liegt somit nicht im Regelungsentwurf, sondern in der Genauigkeit der zugrunde liegenden Positionsdaten.

In der Praxis erreicht GNSS-basierte Landung jedoch nur Meter-Genauigkeit. Patrik u. a. (2019) berichten von einer durchschnittlichen Landeabweichung von 1,11 m; auch beim Open-Source-Autopiloten PX4 beträgt die GNSS-basierte Landegenauigkeit mehrere Meter (Meier u. a. 2015; Wubben u. a. 2019), sodass für höhere Präzision zusätzliche Senso-

ren erforderlich sind (*Precision Landing* 2026). Die Positionsgenauigkeit lässt sich zwar durch RTK-Korrekturen auf Zentimeter-Niveau steigern — Tavasci u. a. (2024) berichten zentimetergenaue Echtzeit-Navigation unter idealen Bedingungen —, allerdings fällt die Genauigkeit in schwierigen GNSS-Umgebungen, etwa bei Signalabschattung durch Vegetation oder Gebäude, auf Meter-Niveau zurück. Gerade in Agrarumgebungen mit Baumkronen und Laubdächern sind solche Bedingungen typisch.

GNSS bleibt damit als robuste Grobpositionierung unverzichtbar, weist jedoch für Präzisionslandung in strukturierten Umgebungen fundamentale Limitationen auf: Die Genauigkeit reicht nicht für das Landen zwischen Baumreihen oder Weinreben mit einem typischen Reihenabstand von 1,5–2 m. Zudem liefert GNSS keinerlei Umgebungswahrnehmung; Veränderungen der Umgebung, Hindernisse oder beengte Platzverhältnisse bleiben unerkannt.

3.3 Markerbasierte bildgestützte Methoden

Bildbasierte Landesysteme mit visuellen Markern bieten eine deutlich höhere Präzision als rein GNSS-gestützte Methoden. So erreicht das ArUco-basierte System von Wubben u. a. (2019) durch deterministische Bildverarbeitung eine durchschnittliche Landeabweichung von 11 cm bei zuverlässiger Markerdetektion aus bis zu 30 m Höhe. Solche klassischen Ansätze setzen jedoch voraus, dass die Markererkennung unter allen Betriebsbedingungen robust funktioniert — eine Annahme, die bei Verdeckung, wechselnden Lichtverhältnissen oder beschädigten Markern nicht gegeben ist.

Reinforcement Learning bietet hier einen konzeptionellen Vorteil: Statt auf handspezifizierte Detektionsalgorithmen angewiesen zu sein, lernt ein RL-Agent eine End-to-End-Steuerung direkt aus Bilddaten und kann dabei implizit Robustheit gegenüber visuellen Störungen erwerben. Polvara u. a. (2020) demonstrieren diesen Ansatz erstmals für die autonome Landung: Ihr System nutzt sequentielle Deep-Q-Networks mit diskretem Aktionsraum und einer Divide-and-Conquer-Strategie, die den Landeprozess in Marker-Ausrichtung und vertikalen Abstieg unterteilt — ein Ansatz, der das Problem dünn besetzter Belohnungssignale adressiert, indem die komplexe Aufgabe in einfachere Teilziele zerlegt wird. Als einziger Sensor dient eine niedrig aufgelöste monokulare Graustufenkamera. Ausschließlich in Simulation mit Domain Randomization trainiert, erzielt das System Erfolgsraten von 89 % für den vertikalen Abstieg, die im realen Flug auf 61 % sinken. Der Robustheitsvorteil gegenüber klassischen Methoden zeigt sich unter erschwerten Bedingungen: Bei verrauschten Markertexturen erreicht das RL-System noch 51 %, während ein konventioneller AR-Tracker vollständig versagt — aller-

dings ist auch dieser Wert für sicherheitskritische Anwendungen unzureichend. Eine zentrale methodische Einschränkung bleibt der diskrete Aktionsraum, der die Steuerungspräzision limitiert.

Ladosz u. a. (2024) erweitern den Ansatz auf bewegliche Landeplattformen und adressieren diese Einschränkung durch einen kontinuierlichen Aktionsraum, der erstmals auch die Vertikalgeschwindigkeit einschließt — bei früheren Arbeiten wird der vertikale Abstieg typischerweise regelbasiert gesteuert. Der Agent erhält neben Graustufenbildern auch IMU-Daten (Geschwindigkeit und Lage) als Zustandsinformation. In der Simulation erreicht das System Erfolgsraten von bis zu 97,4 % bei einer Landeabweichung von 0,52 m; im realen Transfer sinkt die Rate auf rund 70 %. Die Autoren zeigen zudem, dass Domain Randomization zwar notwendig für den Sim-to-Real-Transfer ist, eine zu starke Randomisierung die Performance jedoch signifikant verschlechtert. Die Experimente beschränken sich auf einfache Szenarien ohne Hindernisse und gleichmäßig beleuchtete Innenräume.

Trotz dieser Fortschritte weisen markerbasierte Systeme grundlegende Einschränkungen auf, die unabhängig von der gewählten Steuerungsmethode gelten: Sie setzen vorplatzierte visuelle Infrastruktur am Landeziel voraus, was ihren Einsatz auf vordefinierte Orte begrenzt und in unbekannten Umgebungen unpraktikabel macht. Hinzu kommen methodische Limitationen der RL-basierten Ansätze: Beide Systeme wurden in vereinfachten Simulationsumgebungen ohne Hindernisse trainiert, und der Transfer in die reale Welt geht mit deutlichen Leistungseinbußen einher — Ladosz u. a. (2024) berichten einen Rückgang der Erfolgsrate von 97,4 % auf rund 70 % (vgl. auch Polvara u. a. 2020). Diese Limitationen motivieren die Entwicklung markerfreier Systeme, die Landezonen eigenständig aus Bilddaten ableiten und keine zusätzliche Bodeninfrastruktur erfordern.

3.4 Markerfreie bildgestützte Methoden

Markerfreie Verfahren ermöglichen den Einsatz in unbekannten, unstrukturierten und dynamischen Umgebungen, da sie keine manuelle Vorbereitung der Landezone erfordern — eine Eigenschaft, die insbesondere für Notlandungen oder den Betrieb in unzugänglichen Gebieten entscheidend ist (Yang u. a. 2018; Bartolomei u. a. 2022). Die Bandbreite der Ansätze reicht dabei von klassischen modularen Pipelines bis hin zu End-to-End-Steuerungen mittels Deep Reinforcement Learning.

Yang u. a. (2018) verfolgen einen modularen Ansatz, bei dem aus monokularer SLAM-Rekonstruktion eine 3D-Punktwolke erzeugt wird, aus der anhand von Oberflächensemantik

und Geländehöhe sichere Landezonen in einer Grid-Map identifiziert werden. Das System nutzt ausschließlich Visual-Inertial-Odometrie zur Zustandsschätzung und benötigt weder GNSS noch externe Lokalisierungsinfrastruktur. Es demonstriert Landungen in verschiedenen Umgebungen, weist jedoch aufgrund der hohen Pipeline-Komplexität eine Landezeit von rund zwei Minuten auf.

Einen End-to-End-Ansatz wählen Bartolomei u. a. (2022): Ihr RL-Agent bildet Tiefenbilder und semantische Segmentierungen — sogenannte Mid-Level-Repräsentationen, die generischer als Rohbilder sind und schnelleres Training ermöglichen — direkt auf diskrete Steuerkommandos ab. Das System erreicht in der Simulation Erfolgsraten von über 70 % bei minimaler Sensorausstattung mit lediglich einer monokularen RGB-Kamera. Ein methodisch relevanter Aspekt ist die Sim-to-Real-Strategie: Die Policy wird zunächst mit Ground-Truth-Tiefen- und Semantikdaten trainiert und anschließend mit verrauschten Prädiktionen neuronaler Netze feinabgestimmt, um den Simulationstransfer zu ermöglichen. Im Realflug landete die Policy erfolgreich, während sowohl eine kommerzielle als auch eine State-of-the-Art-Lösung am verrauschten Sensorinput scheiterten. Einschränkend setzt das System voraus, dass semantische Segmentierungen und Tiefenbilder stets verfügbar sind, und der diskrete Aktionsraum (zehn mögliche Aktionen) limitiert die Steuerungsflexibilität.

Dass Deep Reinforcement Learning auch unter dynamischen Umgebungsbedingungen effektive Landesteuerungen ermöglicht, zeigen Peter u. a. (2024) mit TornadoDrone, einem bio-inspirierten DRL-Framework. Bei einfachen Aufgaben erreicht das System Erfolgsraten von 100 %, bei komplexen Aufgaben unter starken Windturbulenzen noch 60 % — insgesamt über 78 % — und übertrifft damit eine konventionelle PID+EKF-Baseline deutlich, die bei komplexen Aufgaben lediglich 10 % erzielt. Allerdings stützen sich die realen Experimente auf ein Vicon-Lokalisierungssystem, das millimetergenaue Positionsdaten liefert und unter Einsatzbedingungen nicht verfügbar ist.

3.5 Tiefenbildbasierte Hindernisvermeidung

Loquercio u. a. (2021) demonstrieren ein End-to-End-Navigationssystem für Hochgeschwindigkeitsflüge durch komplexe, unbekannte Umgebungen mit ausschließlich bordeigener Sensorik und Rechenleistung. Das System nutzt Tiefenbilder einer Stereokamera und IMU-Daten, um kollisionsfreie Kurzzeittrajektorien zu erzeugen, die von einem nachgeschalteten PID-Regler ausgeführt werden. Trainiert wird ausschließlich in Simulation mittels Privileged Learning, einer Imitation eines Experten mit Zugang zu perfekter Zustandsinformation,

wobei lediglich einfache Hindernisgeometrien (Zylinder, Quader, Baumstämme) verwendet werden. Durch die Simulation realistischer Sensorrauschens gelingt ein Zero-Shot-Transfer in reale Umgebungen, die während des Trainings nie erfahren wurden: dichte Wälder, verschneites Gelände und eingestürzte Gebäude. Bei niedrigen Geschwindigkeiten erreicht das System eine Erfolgsrate von 100 % in allen getesteten Umgebungen; im Vergleich zu modularen State-of-the-Art-Planern reduziert der Ansatz die Ausfallrate um den Faktor 10. Eine Einschränkung ist die geringe Positioniergenauigkeit, da Erfolg bereits bei einem Abstand von 5 m zum Ziel definiert wird und der Fokus auf Geschwindigkeit statt Präzision liegt. Methodisch relevant für die vorliegende Arbeit ist der Nachweis, dass ein lernbasierter Ansatz mit Tiefenbildern und hierarchischer Architektur (Policy $\rightarrow$ PID) in dicht bewachsenen natürlichen Umgebungen robust navigieren kann. Die vorliegende Arbeit verfolgt eine vergleichbare Architektur, ersetzt jedoch Privileged Learning durch Reinforcement Learning und verschiebt den Fokus von Hochgeschwindigkeitsnavigation auf Präzisionslandung in beengten Umgebungen.

Kim u. a. (2022) verfolgen einen zweistufigen Ansatz, bei dem zunächst ein vortrainiertes CNN Tiefenbilder aus monokularen RGB-Aufnahmen schätzt und anschließend ein Dueling-Double-DQN auf Basis dieser geschätzten Tiefeninformation diskrete Ausweichentscheidungen trifft. Als einziger externer Sensor dient die bordeigene Monokularkamera der Drohne; Zustandsinformation wie Position oder Geschwindigkeit wird nicht benötigt. Das System wird in Simulation trainiert und ohne weitere Anpassung auf eine reale Drohne übertragen, wo es enge Korridore mit texturlosen Wänden, Personen und Kisten kollisionsfrei durchfliegt. Evaluert wird anhand der zurückgelegten Strecke ohne Kollision; eine gezielte Navigation zu einem definierten Ziel ist nicht Teil des Systems. Die Testumgebungen beschränken sich auf strukturierte Innenräume; Experimente in natürlichen oder vegetationsreichen Umgebungen werden nicht durchgeführt. Der Ansatz demonstriert, dass selbst aus monokularer RGB-Eingabe geschätzte Tiefeninformation für lernbasierte Hindernisvermeidung ausreicht, allerdings mit den Einschränkungen eines diskreten Aktionsraums und der Abhängigkeit von der Qualität der Tiefenschätzung.

Xu u. a. (2024) nutzen direkte Tiefenbilder, um mittels PPO eine kontinuierliche Policy für sichere Navigation in Umgebungen mit statischen und dynamischen Hindernissen zu trainieren. Die Architektur verarbeitet die Tiefenbilder zu einer 3D-Belegungskarte, die zusammen mit Drohnenzustandsdaten (Position, Geschwindigkeit, Orientierung) als Eingabe für das Actor-Critic-Netzwerk dient. Das Training erfolgt parallelisiert in Simulation mit Curricu-

lum Learning, bei dem die Anzahl dynamischer Hindernisse schrittweise erhöht wird. Ein nachgeschalteter Safety Shield auf Basis von Velocity Obstacles ergänzt die gelernte Policy zur Laufzeit. Das System erreicht Zero-Shot-Transfer auf eine reale Drohne und erzielt in Szenarien mit dynamischen Hindernissen die geringste Kollisionsrate im Vergleich zu bestehenden Methoden. Auch hier beschränken sich die Testumgebungen auf strukturierte Indoor-Szenarien mit geometrisch einfachen Hindernissen.

Gemeinsam zeigen diese Arbeiten, dass tiefenbildbasierte lernende Systeme Hindernisse zuverlässig vermeiden können, unabhängig davon ob die Tiefeninformation direkt gemessen (Loquercio u. a. 2021; Xu u. a. 2024) oder aus RGB-Bildern geschätzt wird (Kim u. a. 2022). Allerdings testen alle genannten Systeme in strukturierten Innenräumen oder Waldumgebungen; Evaluationen in landwirtschaftlichen Umgebungen mit Rebzeilen, niedrigen Baumkronen oder dichter Bodenvegetation fehlen.

3.6 Steuerungsarchitekturen im Reinforcement Learning

Die in Abschnitt ?? eingeführte Unterscheidung zwischen modularen und End-to-End-Methoden spiegelt sich auch in der Steuerungsarchitektur der betrachteten RL-Ansätze wider. End-to-End-Systeme wie (Polvara u. a. 2020; Kim u. a. 2022; Ladosz u. a. 2024) erzeugen Bewegungskommandos direkt aus Sensorinput, während hierarchische Ansätze wie (Loquercio u. a. 2021; Bartolomei u. a. 2022) die Policy-Ausgabe an einen nachgeschalteten Regler delegieren. Xu u. a. (2024) kombinieren beide Paradigmen, indem sie eine PPO-Policy mit einem regelbasierten Safety Shield ergänzen.

Die Wahl der Abstraktionsebene beeinflusst die Lernperformance substantiell: **esserActionSpaceDesign2022** zeigen in einer systematischen Studie, dass höherrangige Aktionsräume die Exploration stabilisieren, da der nachgeschaltete Regler ruckartige Policy-Ausgaben glättet. Speziell für Quadrotoren bestätigen **kaufmannBenchmarkComparisonLearned2022**, dass Geschwindigkeitskommandos schneller konvergieren als direktes Body-Rate-Thrust-Control. **aljalboutRoleActionSpace** zeigen zudem, dass höherrangige Aktionsräume einen besseren Sim-to-Real-Transfer ermöglichen, da der Low-Level-Regler Hardware-Abweichungen kompensiert.

Neben der Steuerungsarchitektur hat sich PPO als dominierender Trainingsalgorithmus für RL-basierte Drohnensteuerung etabliert. Sowohl Bartolomei u. a. (2022) (Abschnitt ??) als auch Xu u. a. (2024) (Abschnitt ??) setzen PPO als Trainingsalgorithmus ein. Kaufmann u. a. (2023) demonstrieren den bislang eindrucksvollsten Einsatz: Ihr mit PPO in Simulation

trainiertes System schlägt im physischen Drohnenrennen drei menschliche Weltmeister in direkten Kopf-an-Kopf-Rennen.

Die vorliegende Arbeit folgt dem hierarchischen Paradigma: Der RL-Agent gibt eine Zielposition vor, die ein kaskadierender PID-Regler (Position $\rightarrow$ Geschwindigkeit $\rightarrow$ Lage $\rightarrow$ Drehzahl) in Motorkommandos umsetzt. Die Ausgabe des Agenten bleibt dadurch interpretierbar, der PID-Regler kann unabhängig vom RL-Training auf die Zielplattform abgestimmt werden, und die Lernkomplexität reduziert sich auf die Frage, *wohin* die Drohne fliegen soll.

Zusammenfassend zeigt der Stand der Forschung drei wiederkehrende Limitationen RL-basierter Landesysteme: Erstens operieren die meisten Ansätze mit diskreten Aktionsräumen (Polvara u. a. 2020; Bartolomei u. a. 2022) oder vereinfachen den vertikalen Abstieg auf regelbasierte Steuerung — lediglich Ladosz u. a. (2024) setzen einen kontinuierlichen Aktionsraum mit Vertikalkontrolle ein, beschränken sich jedoch auf hindernisfreie Innenräume. Zweitens trainiert keine der betrachteten Arbeiten in Umgebungen mit Hindernissen, wie sie bei einer Landung zwischen Weinreben oder Baumreihen auftreten. Drittens bleibt der Sim-to-Real-Transfer eine offene Herausforderung: Die Erfolgsraten sinken systematisch beim Übergang in die reale Welt — von 89 % auf 61 % (Polvara u. a. 2020), von 97,4 % auf rund 70 % (Ladosz u. a. 2024) —, und reale Experimente stützen sich auf Hilfsmittel wie Vicon-Lokalisierung (Peter u. a. 2024) oder gezieltes Fine-Tuning (Bartolomei u. a. 2022), die unter Einsatzbedingungen nicht verfügbar sind.

Die Wahl der Sensorausstattung ist dabei ein zentraler Designparameter: Drohnen sind aufgrund ihrer Energie- und Gewichtsbeschränkungen typischerweise nur mit minimaler Sensorik ausgestattet, bestehend aus Kameras und Distanzsensoren (Bartolomei u. a. 2022). Semerikov u. a. (2025) zeigen in ihrer Übersichtsarbeit, dass Vision-Inertial-Kombinationen eine gute Balance zwischen Wahrnehmungsleistung und Systemkomplexität bieten, während leistungsfähigere Sensorsetups entsprechend größere Drohnen mit höherer Lade- und Batteriekapazität erfordern. Gleichzeitig ermöglichen fortschreitende Algorithmen, insbesondere lernbasierte Ansätze, dass auch mit leichtgewichtiger Sensorik anspruchsvolle Aufgaben gelöst werden können, die bisher komplexere Hardware voraussetzten. Die in diesem Kapitel betrachteten Arbeiten bestätigen diesen Trend: Erfolgreiche Hindernisvermeidung und Navigation gelingen mit einer einzelnen Kamera (Kim u. a. 2022; Polvara u. a. 2020; Bartolomei u. a. 2022) oder einer Tiefenkamera mit IMU-Zustandsdaten (Loquercio u. a. 2021; Xu u. a. 2024). Die vorliegende Arbeit folgt diesem Prinzip minimaler Sensorik: Als Eingabe dienen

eine nach unten gerichtete Tiefenkamera und ein Zustandsvektor (Position, Orientierung, Geschwindigkeit), wobei die algorithmische Komplexität des RL-Agenten die begrenzte Sensorausstattung kompensiert.

Die vorliegende Arbeit adressiert die ersten beiden Limitationen: Sie untersucht, ob ein RL-Agent mit kontinuierlichem Aktionsraum und Tiefenbildern in Simulation lernen kann, in beengten Umgebungen mit Hindernissen autonom zu landen. Der Sim-to-Real-Transfer liegt außerhalb des Rahmens dieser Arbeit, die sich auf das Training und die Evaluation in der Simulation konzentriert.

4 Methodik

Dieses Kapitel beschreibt das entwickelte System zur autonomen Drohnenlandung mit Hindernisvermeidung. Die in Abschnitt ?? motivierte hierarchische Steuerungsarchitektur bildet den Ausgangspunkt: Ein RL-Agent trifft Navigationsentscheidungen auf hoher Abstraktionsebene, während ein kaskadierender PID-Regler diese in Motorkommandos umsetzt. Die Aufgabe wird als Markov-Entscheidungsprozess formuliert (vgl. Abschnitt ??), in dem der Agent aus Tiefenbildern und einem Zustandsvektor eine Zielposition ableitet. Dieses bewusst minimalistische Sensorset, bestehend aus einer einzelnen Tiefenkamera und Eigenlageschätzung, folgt dem in Abschnitt ?? identifizierten Trend, algorithmische Komplexität anstelle zusätzlicher Hardware einzusetzen.

Training und Evaluation finden vollständig in Simulation statt. Der GPU-parallelisierte Physiksimulator Genesis erzeugt die für PPO benötigten großen Rollout-Batches aus bis zu 256 gleichzeitigen Umgebungsinstanzen. Ein Sim-to-Real-Transfer liegt außerhalb des Rahmens dieser Arbeit.

Abschnitt ?? gibt zunächst einen Überblick über die Systemarchitektur. Anschließend formalisiert Abschnitt ?? die Aufgabe als MDP mit Zustands-, Aktionsraum, Belohnungsfunktion und Terminierungsbedingungen. Abschnitt ?? beschreibt die CNN-MLP-Netzwerkarchitektur, Abschnitt ?? den PPO-Algorithmus einschließlich der Tanh-Aktionsverteilung. Die physische Trainingsumgebung wird in Abschnitt ?? vorgestellt, gefolgt vom PID-Regler (Abschnitt ??), dem Trainingsprozess (Abschnitt ??) und dem Evaluationsprotokoll (Abschnitt ??).

4.1 Systemüberblick

Das System gliedert sich in zwei Steuerungsebenen (Abbildung ??): Auf der oberen Ebene trifft ein RL-Agent alle 3,0 s eine Navigationsentscheidung. Auf der unteren Ebene setzt ein kaskadierender PID-Regler diese Entscheidung mit 100 Hz in Motordrehzahlen um und stabilisiert die Drohne zwischen zwei Entscheidungszeitpunkten. Das vergleichsweise lange Entscheidungsintervall ist eine bewusste Designentscheidung: Es stellt sicher, dass die Drohne zwischen zwei Entscheidungen zur Ruhe kommt und das Tiefenbild aus einer stabilen Fluglage aufgenommen wird, verhindert, dass der PID-Regler durch zu häufig wechselnde Sollwerte destabilisiert wird, und reduziert die Lernkomplexität, da der Agent weniger Entscheidungen pro Episode treffen muss.

Wahrnehmung.

Der Agent erhält zwei Eingaben: einen normalisierten Zustandsvektor $\mathbf{o}_t \in \mathbb{R}^{17}$, der die relative Position zum Ziel, die Orientierung sowie Linear- und Winkelgeschwindigkeit kodiert (vgl. Abschnitt ??), und ein normalisiertes Tiefenbild $\mathbf{D}_t \in [0, 1]^{64 \times 64}$ einer vertikal nach unten gerichteten Kamera mit 90° Sichtfeld. Der Zustandsvektor liefert die Eigenwahrnehmung der Drohne, das Tiefenbild die einzige Information über Hindernisse. Beide Größen werden direkt und unverrauscht aus der Simulation erhoben.

Entscheidung.

Zustandsvektor und Tiefenbild werden der Actor-Critic-Architektur übergeben (vgl. Abschnitt ??). Ein geteilter CNN-Encoder extrahiert Hindernismerkmale aus dem Tiefenbild; der resultierende Merkmalsvektor wird mit dem normalisierten Zustandsvektor konkateniert und an die MLP-Köpfe von Actor und Critic weitergegeben. Der Actor sampelt eine vierdimensionale Aktion $\mathbf{a}_t \in [-1, 1]^4$, die in einen Positionsversatz und einen Sollgierwinkel skaliert wird (vgl. Abschnitt ??).

Ausführung.

Der kaskadierende PID-Regler (vgl. Abschnitt ??) übernimmt die skalierten Sollwerte und regelt die Drohne über 300 Simulationsschritte (3,0 s) zum Sollzustand. Anschließend erhebt die Simulation den neuen Zustand, und der Zyklus beginnt erneut. Der Agent muss somit ausschließlich lernen, *wohin* die Drohne fliegen soll; die Flugstabilisierung übernimmt der Regler.

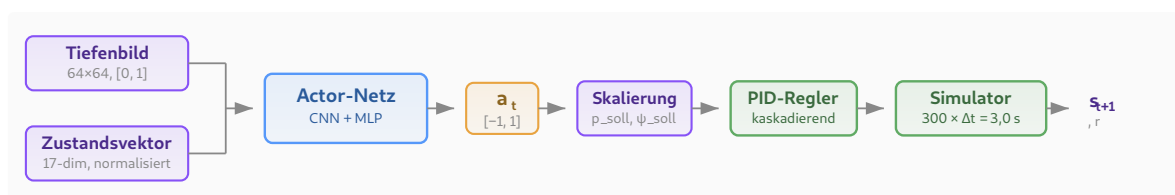


Abbildung 4.1.: Systemarchitektur mit zwei Steuerungsebenen: Der RL-Agent (oben) erhält Tiefenbild und Zustandsvektor, sampelt eine Aktion und gibt Sollwerte vor. Der PID-Regler (unten) setzt diese über 300 Simulationsschritte in Motordrehzahlen um.

4.2 Formulierung als Markov-Entscheidungsprozess

Die autonome Drohnenlandung in einem vertikalen Korridor mit Hindernissen wird als endlicher, episodischer Markov-Entscheidungsprozess (MDP) formalisiert (vgl. Abschnitt ??). Das MDP ist durch das Tupel $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$ definiert, wobei $\mathcal{S}$ den Zustandsraum, $\mathcal{A}$ den Aktionsraum, p die Übergangsdynamik, r die Belohnungsfunktion und $\gamma = 0,99$ den Diskontierungsfaktor bezeichnen.

4.2.1 Zustandsraum

Der Zustand $s_t \in \mathcal{S}$ zum Entscheidungszeitpunkt t setzt sich aus einem numerischen Zustandsvektor $\mathbf{o}_t \in \mathbb{R}^{17}$ und einem Tiefenbild $\mathbf{D}_t \in [0, 1]^{64 \times 64}$ zusammen:

$$s_t = (\mathbf{o}_t, \mathbf{D}_t). \quad (4.1)$$

Der Zustandsvektor

$$\mathbf{o}_t = (\tilde{\mathbf{p}}_{\text{rel},t}, \mathbf{q}_t, \tilde{\mathbf{v}}_t^B, \tilde{\omega}_t^B, \mathbf{a}_{t-1}) \quad (4.2)$$

enthält die in Tabelle ?? aufgeschlüsselten Größen. Die Tilde kennzeichnet skalierte und auf $[-1, 1]$ geklippte Werte; die Skalierungsfaktoren normieren die physikalischen Bereiche auf eine einheitliche Größenordnung.

Tabelle 4.1.: Komponenten des Zustandsvektors $\mathbf{o}_t \in \mathbb{R}^{17}$.

Größe	Dim.	Beschreibung	Skalierung
$\tilde{\mathbf{p}}_{\text{rel},t}$	3	Relative Position zum Ziel	$\times \frac{1}{15}, \in [-1, 1]$
$\mathbf{q}_t$	4	Orientierungsquaternion (w, x, y, z)	—
$\tilde{\mathbf{v}}_t^B$	3	Lineargeschwindigkeit	$\times 0,4, \in [-1, 1]$
$\tilde{\omega}_t^B$	3	Winkelgeschwindigkeit	$\times \frac{1}{\pi}, \in [-1, 1]$
$\mathbf{a}_{t-1}$	4	Aktion des vorherigen Zeitschritts	—

Die relative Position $\tilde{\mathbf{p}}_{\text{rel},t} = \min(\max(\frac{1}{15}(\mathbf{p}_{\text{Ziel}} - \mathbf{p}_t), -1), 1)$ kodiert Abstand und Richtung zum Landeziel, wobei der Skalierungsfaktor auf die maximale Diagonaldistanz zum Ziel ausgelegt ist. Die Lineargeschwindigkeit wird mit dem Faktor 0,4 skaliert, sodass der typische Geschwindigkeitsbereich der Drohne auf $[-1, 1]$ abgebildet wird; die Winkelgeschwindigkeit wird durch π dividiert und erreicht damit bei einer vollen Drehung pro Sekunde den Sättigungswert.

Das Tiefenbild $\mathbf{D}_t \in [0, 1]^{64 \times 64}$ wird von einer am Drohnenkörper vertikal nach unten gerichteten Kamera mit 90° Sichtfeld erzeugt:

$$\mathbf{D}_t = \min \left(\max \left(\frac{\mathbf{d}_{\text{roh}}}{d_{\text{max}}}, 0 \right), 1 \right), \quad d_{\text{max}} = 20 \text{ m.} \quad (4.3)$$

Das Tiefenbild liefert die einzige Information über Lage und Geometrie der Hindernisse; der numerische Zustandsvektor enthält keine explizite Hindernisrepräsentation.

4.2.2 Aktionsraum

Der Aktionsraum ist vierdimensional und kontinuierlich:

$$\mathbf{a}_t = (a_x, a_y, a_z, a_\psi) \in [-1, 1]^4. \quad (4.4)$$

Die ersten drei Komponenten definieren einen Positionsversatz relativ zur aktuellen Drohnenposition:

$$\mathbf{p}_{\text{soll}} = \mathbf{p}_t + \begin{pmatrix} a_x \cdot \sigma_x \\ a_y \cdot \sigma_y \\ a_z \cdot \sigma_z \end{pmatrix}, \quad \boldsymbol{\sigma} = (1,0; 1,0; 2,0) \text{ m.} \quad (4.5)$$

Die erhöhte z -Skala ermöglicht größere vertikale Schritte, die für den vorwiegend vertikalen Abstieg erforderlich sind. Die vierte Komponente legt den Sollgierwinkel fest:

$$\psi_{\text{soll}} = a_\psi \cdot 180^\circ. \quad (4.6)$$

Die Sollwerte $(\mathbf{p}_{\text{soll}}, \psi_{\text{soll}})$ werden nicht direkt an die Motoren übergeben, sondern von einem kaskadierenden PID-Regler in Motordrehzahlen umgesetzt (vgl. Abschnitt ??). Diese Entkopplung reduziert den Aktionsraum von vier Motordrehzahlen auf vier semantisch interpretierbare Navigationskommandos.

4.2.3 Übergangsmodell

Die Übergangsdynamik $p(s_{t+1} \mid s_t, \mathbf{a}_t)$ wird deterministisch durch den Physiksimulator bestimmt. Der Simulator integriert die Physik mit einer Schrittweite von $\Delta t = 0,01 \text{ s}$ (100 Hz). Der RL-Agent trifft nicht bei jedem Simulationsschritt eine Entscheidung, sondern alle 300 Schritte, entsprechend einem Entscheidungsintervall von 3,0 s. Zwischen zwei Entscheidungen steuert ein kaskadierender PID-Regler die Drohne zum Sollzustand $(\mathbf{p}_{\text{soll}}, \psi_{\text{soll}})$.

4.2.4 Belohnungsfunktion

Eine zentrale Herausforderung beim Einsatz von Deep Reinforcement Learning in der Robotik liegt in der Spärlichkeit der Belohnungssignale: Der Agent muss große Zustands- und Aktionsräume explorieren, erhält jedoch nur selten informatives Feedback (Polvara u. a. 2020). Die Belohnungsfunktion kombiniert daher dichte Signale, die den Fortschritt zur Zielerreichung messen, mit spärlichen terminalen Signalen, die das eigentliche Ziel kodieren. Die Fortschrittskomponente ist dabei als potentialbasiertes Reward Shaping im Sinne von Ng u. a. (1999) konstruiert ($\Phi(s) = -d(s)$); die übrigen dichten Komponenten weichen bewusst davon ab, da in Vorversuchen eine rein potentialbasierte Belohnungsfunktion zu langsamerer Konvergenz führte.

Für die Aufgabe des autonomen Landens mit Hindernisvermeidung wurden folgende Entwurfsprinzipien berücksichtigt:

- Die Drohne soll sicher fliegen und Hindernisse meiden.
- Die Drohne soll den kürzesten Weg zum Ziel nehmen.
- Die Drohne muss in der Nähe des Ziels zum Stillstand kommen.

Die resultierende Belohnungsfunktion setzt sich als gewichtete Summe aus fünf Komponenten zusammen:

$$r_t = w_{\text{prog}} r_{\text{prog}} + w_{\text{nah}} r_{\text{nah}} + w_{\text{prox}} r_{\text{prox}} + w_{\text{crash}} r_{\text{crash}} + w_{\text{erf}} r_{\text{erf}} \quad (4.7)$$

Dabei wird jede Komponente am Übergang (s_t, a_t, s_{t+1}) berechnet; $d_t = \|\mathbf{p}_{\text{ziel}} - \mathbf{p}_t\|$ bezeichnet den euklidischen Abstand zum Ziel vor und d_{t+1} den Abstand nach Ausführung der Aktion.

Fortschritt. Die Fortschrittsbelohnung misst die Annäherung an das Ziel zwischen zwei aufeinanderfolgenden Zeitschritten:

$$r_{\text{prog}} = d_t - d_{t+1}. \quad (4.8)$$

Ein positiver Wert zeigt an, dass sich die Drohne dem Ziel genähert hat. Bei direktem Abstieg zum Ziel beträgt $r_{\text{prog}} \approx 2,0$ pro Zeitschritt.

Nähe. Die Nähebelohnung liefert ein exponentiell ansteigendes Signal, das in der Nähe des Ziels signifikant wird:

$$r_{\text{nah}} = \exp(-d_{t+1}). \quad (4.9)$$

Bei Abständen über ca. 5 m ist r_{nah} vernachlässigbar ($< 0,01$); am Ziel nähert sich der Wert 1. Die Komponente belohnt das Verbleiben in der Nähe des Ziels.

Hindernisproximität. Die Hindernisproximität bestraft das Eindringen in einen Sicherheitsradius $d_s = 3,0$ m um Hindernisse:

$$r_{\text{prox}} = \max\left(1 - \frac{d_{\min}}{d_s}, 0\right), \quad (4.10)$$

wobei $d_{\min}$ den minimalen Abstand zur achsenparallelen Begrenzungsbox (AABB) des nächsten Hindernisses bezeichnet. Die Strafe steigt linear mit abnehmendem Abstand. Der Sicherheitsradius wurde empirisch ermittelt.

Absturz. Die Absturzstrafe wird bei Verletzung einer Abbruchbedingung (vgl. Abschnitt ??) ausgelöst:

$$r_{\text{crash}} = \begin{cases} 1 & \text{Hinderniskollision, Bodenkollision, Kontrollverlust} \\ & \text{oder Verlassen des Korridors,} \\ 0 & \text{sonst.} \end{cases} \quad (4.11)$$

Erfolg. Der Erfolgsbonus wird bei Erreichen des Landeziels vergeben:

$$r_{\text{erf}} = \begin{cases} 1 & \text{Landung innerhalb des Zielradius,} \\ 0 & \text{sonst.} \end{cases} \quad (4.12)$$

Die Gewichte wurden experimentell in mehreren Trainingsdurchläufen abgestimmt. Eine zu hohe Gewichtung der Hindernisproximität und der Absturzstrafe führt zu einer stagnierenden Policy: Da Annäherung an das Ziel und erfolgreiche Landungen zu Beginn des Trainings selten und zufällig auftreten, dominieren die negativen Belohnungen, und der Agent lernt, auf der Stelle zu schweben statt abzusteigen. Erst eine Abschwächung dieser Strafen ermöglicht exploratives Verhalten. Umgekehrt mussten Fortschritt und Nähe gegenüber der Erfolgsbelohnung ausbalanciert werden, damit der Agent nicht dauerhaftes Fliegen gegenüber dem Beenden der Episode bevorzugt. Auf eine explizite Timeout-Strafe wurde bewusst verzichtet: Ohne eine Zeitbeobachtung im Zustandsvektor kann der Critic den Episodenabbruch nicht antizipieren, was ohne Anpassung des PPO-Algorithmus zu schlechterer Performance führt (Rudin u. a. 2022).

Tabelle ?? fasst die gewählten Gewichte zusammen.

Tabelle 4.2.: Gewichte der Belohnungskomponenten. Negative Gewichte kennzeichnen Strafterme.

Komponente	Symbol	Gewicht
Fortschritt	w_{prog}	1,0
Nähe	w_{nah}	2,0
Hindernisproximität	w_{prox}	-0,2
Absturz	w_{crash}	-10,0
Erfolg	w_{erf}	20,0

4.2.5 Terminierung

Eine Episode endet, wenn eine der folgenden Bedingungen eintritt:

Erfolg. Die Drohne verbleibt über die gesamte Dauer eines Entscheidungsschritts (3,0 s) innerhalb eines Radius von 0,5 m um das Landeziel.

Absturz. Mindestens eine der folgenden Bedingungen ist erfüllt:

- Flughöhe $z < 0,2$ m (Bodenkollision),
- Neigung $> 60^\circ$ in Roll oder Pitch (Kontrollverlust),
- AABB-Abstand zum nächsten Hindernis $< 0,3$ m (Hinderniskollision),
- Verlassen des Korridors (Bounding-Box-Prüfung auf allen Achsen).

Timeout. Die maximale Episodenlänge von $T_{\text{max}} = 20$ Entscheidungsschritten (60 s) ist erreicht. Timeouts werden im Vorteilsschätzer als nicht-terminale Übergänge behandelt, sodass die Wertfunktion den erwarteten Ertrag über die Episode hinaus approximiert.

4.3 Netzwerkarchitektur

PPO verwendet eine Actor-Critic-Architektur (vgl. Abschnitt ??), in der zwei Netzwerke parallel trainiert werden: Der *Actor* π_θ gibt eine stochastische Strategie aus, der *Critic* V_ϕ schätzt die State-Value-Funktion zur Berechnung des Vorteilsschätzers.

Der CNN-Encoder besteht aus drei Faltungsschichten mit aufsteigender Kanalzahl (32, 64, 128), abnehmenden Kernelgrößen (8×8 , 4×4 , 3×3) und Schrittweiten (4, 2, 1). Nach jeder Faltungsschicht folgt eine Batch-Normalisierung und eine ELU-Aktivierung. Ein abschließendes Global Max Pooling reduziert jede der 128 Feature-Maps auf einen Skalar und erzeugt so einen 128-dimensionalen Merkmalsvektor unabhängig von der räumlichen Auflösung der Eingabe. Der Zustandsvektor durchläuft vor der Konkatination eine laufende empirische Normalisierung, die Mittelwert und Varianz über alle bisherigen Beobachtungen schätzt und die heterogenen physikalischen Größen auf einheitliche Skala bringt. Der resultie-

rende Vektor aus CNN-Merkmalen und normalisiertem Zustand wird an die MLP-Köpfe von Actor und Critic weitergegeben, die jeweils aus drei vollvernetzten Schichten mit 128 Neuronen und ELU-Aktivierung bestehen. Actor und Critic teilen sich den CNN-Encoder; die MLP-Köpfe werden getrennt trainiert. Der Actor gibt pro Aktionsdimension einen Mittelwert μ und eine lernbare Log-Standardabweichung $\log \sigma$ aus und parametrisiert damit eine Tanh-Gaußverteilung, deren Motivation und Formulierung in Abschnitt ?? erläutert werden. Der Critic gibt einen skalaren State-Value $V(s_t)$ aus. Abbildung ?? zeigt die vollständige Architektur mit allen Schichten.

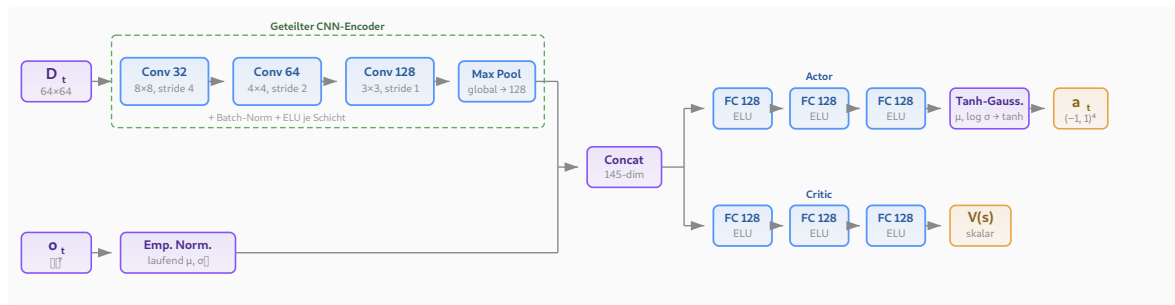


Abbildung 4.2.: Netzwerkarchitektur: Das Tiefenbild durchläuft einen geteilten CNN-Encoder (drei Faltungsschichten mit Batch-Normalisierung, ELU und abschließendem Global Max Pooling). Der resultierende 128-dimensionale Merkmalsvektor wird mit dem empirisch normalisierten Zustandsvektor konkateniert und an die getrennten MLP-Köpfe von Actor und Critic weitergegeben.

4.4 Algorithmus

4.4.1 Proximal Policy Optimization

Als Trainingsalgorithmus wird *Proximal Policy Optimization* (PPO) (Schulman, Wolski u. a. 2017) eingesetzt, dessen Clipped Surrogate Objective und vollständige Zielfunktion in Abschnitt ?? eingeführt wurden.

Die vorliegende Arbeit zielt auf einen kontinuierlichen Aktionsraum ab, in dem der Agent Zielpositionsoffsets und eine Gierrate ausgibt (vgl. Abschnitt ??). Wie in Abschnitt ?? dargestellt, übertrifft PPO auf strukturell vergleichbaren kontinuierlichen Steuerungsaufgaben die Vergleichsverfahren TRPO und A2C. Auch im spezifischen Anwendungsfeld der Drohnensteuerung hat sich PPO als Standardverfahren etabliert: Sowohl Bartolomei u. a. (2022) und Xu u. a. (2024) als auch Kaufmann u. a. (2023) setzen PPO ein (vgl. Abschnitt ??). PPO passt zudem architektonisch zum gewählten Trainingssetup: Als On-Policy-Verfahren verarbeitet es die gesammelten Daten direkt und verwirft sie nach dem Update. Off-Policy-Verfahren wie SAC (Haarnoja u. a. 2018) wurden daher verworfen. Sie erfordern einen Replay Buffer, der vergangene Transitionen speichert und bei jedem Update erneut abtastet. Ihre höhere Sample-

Effizienz setzt zudem voraus, dass die Datensammlung der Engpass ist. Bei GPU-paralleler Simulation mit hunderten bis tausenden Umgebungen ist jedoch der Gradientenupdate der Engpass; die geringere Sample-Effizienz von On-Policy-Methoden wird durch den Datendurchsatz mehr als kompensiert (Rudin u. a. 2022). Neuere PPO-Varianten wie Phasic Policy Gradient (Cobbe u. a. 2021) wurden nicht evaluiert, da die verwendete Trainingsumgebung rsl-rl (Rudin u. a. 2022) ausschließlich die Standard-PPO-Implementierung bereitstellt.

4.4.1.1 Aktionsverteilung und Tanh-Squashing

Der Actor parametrisiert eine Gaußverteilung über einem unbeschränkten Hilfsraum: Für jede Aktionsdimension i wird ein Rohwert $u_i \sim \mathcal{N}(\mu_i, \sigma_i^2)$ gesampelt, wobei μ_i und $\log \sigma_i$ Netzausgaben sind. Der Aktionsraum ist jedoch auf $[-1, 1]^4$ beschränkt (vgl. Abschnitt ??). Ein naives Clipping $a_i = \text{clip}(u_i, -1, 1)$ würde die Aktionswerte korrekt begrenzen, die zugehörige Log-Wahrscheinlichkeit jedoch weiterhin auf dem unbeschnittenen Wert u_i berechnen. Da PPO das Verhältnis $r_t(\theta) = \pi_\theta(\mathbf{a}_t \mid s_t) / \pi_{\theta_{\text{alt}}}(\mathbf{a}_t \mid s_t)$ zur Gradientenschätzung nutzt (vgl. Abschnitt ??), führt diese Diskrepanz zwischen ausgeführter und bewerteter Aktion zu verzerrten Gradienten.

Statt Clipping wird daher eine invertierbare, differenzierbare Transformation verwendet (Haarnoja u. a. 2018):

$$a_i = \tanh(u_i). \quad (4.13)$$

Die zugehörige Log-Wahrscheinlichkeit ergibt sich über die Substitutionsregel für Wahrscheinlichkeitsdichten (Change of Variables):

$$\log \pi(\mathbf{a} \mid s) = \log \mathcal{N}(\mathbf{u} \mid \boldsymbol{\mu}, \boldsymbol{\sigma}^2) - \sum_{i=1}^4 \log(1 - \tanh^2(u_i)). \quad (4.14)$$

Der Korrekturterm $-\sum_i \log(1 - \tanh^2(u_i))$ ist die Log-Jacobi-Determinante der Tanh-Transformation und stellt sicher, dass die Dichte im transformierten Raum korrekt normiert bleibt.

Zusätzlich erzwingt ein Floor $\log \sigma_i \geq -2,0$ ($\sigma_i \geq 0,135$) eine Mindestexploration. Ohne diesen Floor kann PPO die Standardabweichung gegen null treiben, was in der Log-Wahrscheinlichkeit zu numerischen Instabilitäten führt und die Strategie irreversibel kollabieren lässt.

4.4.1.2 Hyperparameter

Tabelle ?? fasst die PPO-Hyperparameter zusammen.

Tabelle 4.3.: PPO-Hyperparameter der vorliegenden Arbeit.

Parameter	Wert
Clipping ε	0,2
Diskontierung γ	0,99
GAE λ	0,95
Lern-Epochen K	5
Mini-Batches M	4
c_1 (Value-Loss)	1,0
c_2 (Entropie)	0,003
Gradient-Clipping	1,0

Die Werte für ε , γ , λ sowie K und Gradient-Clipping entsprechen den von Schulman, Wolski u. a. (2017) empfohlenen Standardwerten, die sich in den MuJoCo-Benchmarks als robust über verschiedene Aufgaben erwiesen haben. Der GAE-Parameter $\lambda = 0,95$ folgt der Empfehlung von Schulman, Moritz u. a. (2016) als Kompromiss zwischen Bias und Varianz. Der Entropie-Koeffizient $c_2 = 0,003$ wurde bewusst niedrig gewählt, da die Tanh-Verteilung (vgl. Abschnitt ??) empfindlich auf hohe Entropie-Anreize reagiert; $c_2 \geq 0,01$ führte in Pilotexperimenten zu instabilem Training.

4.5 Trainingsumgebung

4.5.1 Physiksimulator

Das Training eines RL-Agenten vorliegende Aufgabe erfordert einen Physiksimulator, der drei zentrale Anforderungen erfüllt: (1) GPU-parallelisierte Ausführung hunderter Umgebungsinstanzen, um die von PPO benötigten großen Rollout-Batches effizient zu erzeugen (vgl. Abschnitt ??); (2) simulierte Tiefenkameras, da der Agent Tiefenbilder als Beobachtung erhält; (3) Import beliebiger 3D-Objekte, um realitätsnahe Hindernisszenarien zu konstruieren.

Die vorliegende Arbeit verwendet Genesis (Genesis Authors 2024), einen 2024 veröffentlichten, quelloffenen Physiksimulator für Robotik. Genesis erfüllt alle drei definierten Anforderungen: Der Simulator führt bis zu 4096 Umgebungen GPU-parallel aus, stellt Tiefenkameras über eine Batch-Rendering-Pipeline bereit, importiert beliebige 3D-Meshes (u. a. URDF, OBJ) und bietet eine durchgängige Python-API. Genesis zeichnet sich durch eine geringere Einstiegskomplexität aus: Die Installation beschränkt sich auf ein einzelnes Python-Paket, und die API erfordert keine herstellerspezifische Toolchain.

Gegenüber verbreiteten Alternativen bietet Genesis entscheidende Vorteile: Gazebo (Koenig und Howard 2004) und PyBullet (Coumans und Bai 2016–2021) unterstützen keine

GPU-parallele Ausführung hunderter Umgebungen und scheiden daher für datenintensives RL-Training aus. Isaac Gym (Makoviychuk u. a. 2021) erfüllt zwar alle genannten Anforderungen, ist jedoch proprietär an NVIDIA gebunden und erfordert einen umfangreichen Installationsstack.

Einschränkend ist anzumerken, dass Genesis mit seiner Erstveröffentlichung im Dezember 2024 ein vergleichsweise junges Projekt ist und über eine kleinere Community als etablierte Alternativen verfügt.

4.5.2 Drohnenmodell

Die Drohne wird als URDF-Modell mit vier Rotoren simuliert. Die wesentlichen physikalischen Parameter sind eine Masse von 0,714 kg, ein Schub-Gewicht-Verhältnis von 2,25 sowie eine Hover-Drehzahl von 1789 RPM bei einer maximalen Drehzahl von 2700 RPM.

4.5.3 Korridorgeometrie

Die Trainingsumgebung bildet einen vertikalen Korridor der Ausdehnung $5 \times 2 \times 15$ m ($x \times y \times z$) ab (Abbildung ??). Der Korridor besitzt keine physischen Wände; stattdessen wird das Verlassen des Begrenzungsrahmens als Absturz gewertet (vgl. Abschnitt ??). Die Bodenebene befindet sich bei $z = 0$ m.

Die Drohne wird zu Episodenbeginn auf einer zufälligen Position im oberen Korridorbereich bei $z = 12$ m initialisiert, wobei die horizontale Position gleichverteilt innerhalb von $\pm 1,5$ m (x) bzw. $\pm 0,5$ m (y) um die Korridormitte variiert. Die Initialorientierung ist die Nulllage (Quaternion $(1, 0, 0, 0)$). Das Landeziel ist fixiert bei $\mathbf{p}_{\text{ziel}} = (0, 0, 1)^\top$ m.

Während des Abstiegs durchquert die Drohne zwei horizontale Hindernisschichten bei $z \approx 2$ m und $z \approx 7$ m (vgl. Abschnitt ??).

4.5.4 Hindernisse

Als Hindernisse dienen sechs 3D-Objekte aus dem Google Scanned Objects Datensatz (GSO) (Downs u. a. 2022): eine Tasse, ein Teddybär, ein Spielzeugdinosaurier, ein Spielzeugbus, ein Spielzeugnashorn und ein Schuh. Jedes Objekt wird um den Faktor 21 bis 63 skaliert, sodass seine horizontale Ausdehnung mindestens 2 m beträgt und damit die gesamte y -Breite des Korridors abdeckt. Objekte, deren Höhe nach der Skalierung 2,0 m überschreitet, werden von unten beschnitten.

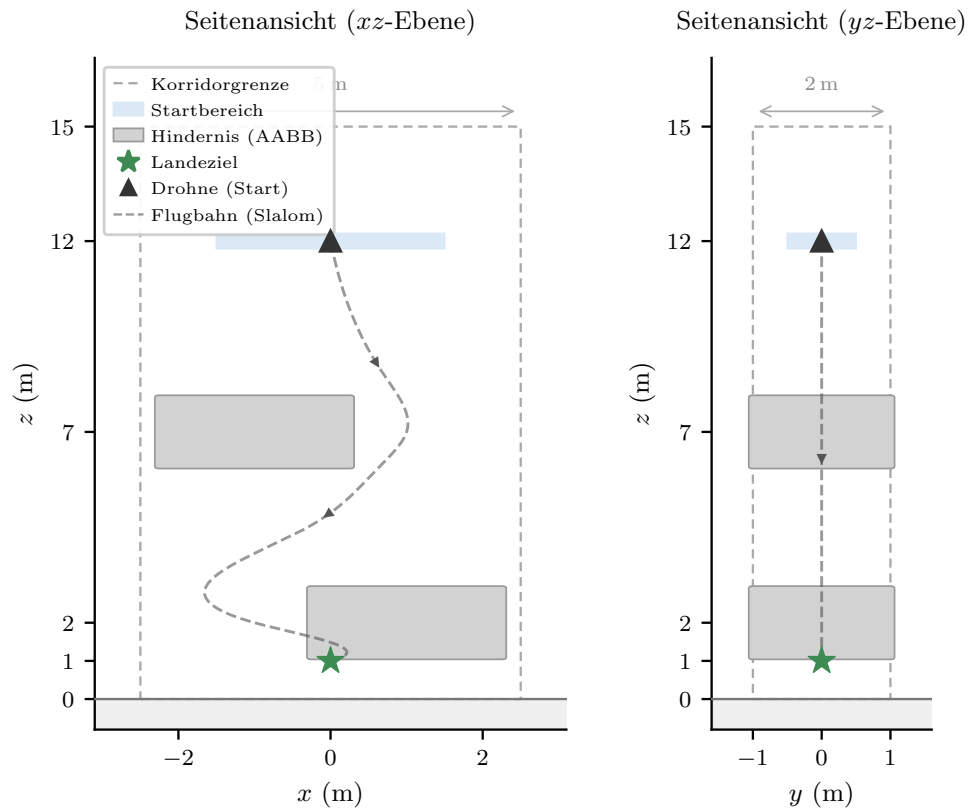


Abbildung 4.3.: Seitenansichten der Korridorgeometrie. Links: xz -Ebene (Slalom-Achse) mit schematischer Flugbahn. Rechts: yz -Ebene. Die gestrichelten Rechtecke markieren die Korridorgrenzen, graue Flächen die schematischen Hindernis-Begrenzungsboxen.

Für die Kollisionserkennung und die Hindernisproximität (vgl. Abschnitt ??) wird die achsenparallele Begrenzungsbox (AABB) jedes Objekts herangezogen. Diese konservative Approximation umschließt das gesamte Mesh vollständig, sodass die Drohne bereits vor Berührung der tatsächlichen Oberfläche als kollidiert gilt.

Die Platzierung folgt einem erzwungenen Slalom entlang der x -Achse: In jeder der zwei Hindernisschichten wird ein zufällig gewähltes Objekt auf einer Seite der Korridormitte positioniert, mit einem Versatz von 0,6 bis 1,2 m. Aufeinanderfolgende Schichten wechseln die Seite, sodass die Drohne einen Slalomkurs fliegen muss. Die Startseite ($+x$ oder $-x$) wird pro Episode zufällig gewählt.

4.6 Kaskadierender PID-Regler

Wie in Abschnitt ?? dargelegt, folgt die vorliegende Arbeit einer hierarchischen Steuerungsarchitektur: Der RL-Agent gibt Sollposition und Sollgierwinkel vor, ein nachgeschalteter Regler setzt diese in Motordrehzahlen um. Der Regler ist als dreistufige PID-Kaskade aufgebaut:

1. **Positionsregler** (äußere Schleife): Vergleicht die Sollposition mit der aktuellen Position und gibt Sollgeschwindigkeiten aus, begrenzt auf 5 m/s horizontal und 3 m/s vertikal.
2. **Geschwindigkeitsregler** (mittlere Schleife): Wandelt Geschwindigkeitsfehler in ein Schubkommando sowie gewünschte Roll- und Nickwinkel um, begrenzt auf $\pm 30^\circ$.
3. **Lageregler** (innere Schleife): Regelt Roll, Nick und Gier auf die gewünschten Winkel und gibt Korrektursignale an den Motormischer.

Der Motormischer addiert die vier Korrektursignale (Schub, Roll, Nick, Gier) zur Schwebedrehzahl 1789 RPM und verteilt sie auf die vier Motoren. Der Agent muss somit ausschließlich die Navigationsstrategie erlernen, nicht die Flugdynamik. Der Regler wurde eigens für diese Simulationsumgebung entwickelt und seine Parameter manuell abgestimmt (Tabelle ??). Ein etablierter Flugreglerstack wie PX4 stand im verwendeten Simulator nicht zur Verfügung.

4.7 Trainingsprozess

Tabelle ?? fasst die Trainingsparameter zusammen.

Tabelle 4.4.: Trainingsparameter.

Parameter	Wert
Parallele Umgebungen N	256
Schritte pro Umgebung T	60
Batch-Größe $N \times T$	15 360
Lernrate α	5×10^{-4}
Optimizer	Adam
Trainingsiterationen	<i>TODO</i>
GPU	NVIDIA A30 (24 GB)
Trainingszeit (Wallclock)	<i>TODO</i>
Checkpoint-Intervall	100 Iterationen

Das Training wird auf dem HPC-Cluster der Universität Leipzig durchgeführt. Als GPU dient eine NVIDIA A30 (Ampere-Architektur, 24 GB VRAM). Die A30 wurde gegenüber der alternativ verfügbaren NVIDIA V100 (Volta) gewählt, da der Batch-Renderer für die Tiefenkamera eine GPU mit Compute Capability $\geq 7,5$ (Turing oder neuer) voraussetzt. Auf der V100 steht lediglich der langsamere Rasterizer-Pfad zur Verfügung, der die Tiefenbilder seriell pro Umgebung rendert.

Die Anzahl paralleler Umgebungen $N = 256$ ist durch den GPU-Speicher begrenzt. Die Rollout-Länge $T = 60$ Entscheidungsschritte bei 3,0 s Entscheidungsintervall (vgl. Abschnitt ??) ergibt eine Rollout-Dauer von 180 s, die drei vollständige Episoden ($T_{\max} = 60$ s) umfassen kann. Pro Iteration sammelt der Agent damit $N \times T = 15\,360$ Transitionen, die

in $M = 4$ Mini-Batches à 3 840 Transitionen aufgeteilt und $K = 5$ Epochen lang optimiert werden.

Die Lernrate $\alpha = 5 \times 10^{-4}$ wurde experimentell bestimmt: Kleinere Werte führten zu langsamer Konvergenz, größere zu kollabierenden Strategien.

Zu Beginn jeder Trainingsausführung werden die Episodenlängen der N Umgebungen zufällig initialisiert, sodass die Episodenenden über die Umgebungen hinweg zeitlich versetzt auftreten. Ohne diese Dekorrelation würden alle Umgebungen synchron zurückgesetzt, was innerhalb eines Rollouts zu korrelierten Transitionsblöcken führt und die Varianz der Gradientenschätzung erhöht (Rudin u. a. 2022).

Der Agent wird über *TODO* Iterationen trainiert, was $TODO \times 15\,360 = TODO$ gesammelten Transitionen entspricht. Alle 100 Iterationen wird ein Checkpoint gespeichert. Die Wallclock-Trainingszeit beträgt *TODO* auf der genannten Hardware.

4.8 Evaluation

Die Evaluation untersucht die Leistungsfähigkeit der trainierten Strategie anhand zweier Versuchsreihen mit jeweils 1 000 Episoden und vergleicht sie mit einem RRT*-Pfadplaner. Die erste Versuchsreihe verwendet die Trainingsumgebung mit zuvor unbekannten Hindernisformen; die zweite eine photogrammetrisch erfasste Agrarumgebung.

4.8.1 Evaluationsmetriken

Jede Episode wird anhand der folgenden Metriken bewertet. Alle Metriken werden mit 95 %-Bootstrap-Konfidenzintervallen versehen (Efron und Tibshirani 1994): Aus den $n = 1\,000$ Episodenergebnissen werden $B = 10\,000$ Stichproben mit Zurücklegen gezogen; für jede Stichprobe wird die jeweilige Metrik berechnet. Die Grenzen des Konfidenzintervalls ergeben sich als das 2,5 %- und 97,5 %-Perzentil der Bootstrap-Verteilung (Perzentilmethode).

Erfolgsrate. Anteil der Episoden, in denen die Drohne das Landeziel innerhalb des Zielradius (0,5 m) erreicht, ohne eine Abbruchbedingung (vgl. Abschnitt ??) auszulösen und innerhalb des Zeitlimits von 60 s.

Fehlermodusverteilung. Jede fehlgeschlagene Episode wird einer von vier Kategorien zugeordnet: Hinderniskollision, Bodenkollision, Verlassen des Korridors (nur Versuchsreihe 1) oder Timeout. Die Verteilung gibt Aufschluss darüber, *warum* die Strategie versagt, nicht nur *ob*.

Landezeit. Zeit vom Episodenbeginn bis zur erfolgreichen Landung, berechnet ausschließlich über erfolgreiche Episoden. Die Landezeit wird als Median mit Interquartilsabstand (IQR) angegeben, da die Verteilung typischerweise rechtsschief ist: Die Mehrzahl der Episoden endet schnell, während vereinzelte Episoden mit ungünstigen Hinderniskonfigurationen deutlich länger dauern. Der Median ist gegenüber solchen Ausreißern robuster als der Mittelwert.

Minimaler Hindernisabstand. Der kleinste AABB-Abstand zwischen Drohne und einem Hindernis, gemessen über den gesamten Episodenverlauf. Angegeben werden der Mittelwert über alle Episoden sowie das globale Minimum. Diese Metrik quantifiziert die Sicherheitsmargen der Strategie und ist auch für erfolgreiche Episoden aussagekräftig: Ein niedriger mittlerer Mindestabstand deutet auf eine riskante Flugweise hin, selbst wenn keine Kollision auftritt.

Pfادهffizienz. Das Verhältnis der tatsächlichen Trajektorienlänge zur euklidischen Distanz zwischen Startposition und Landeziel:

$$\eta_{\text{Pfad}} = \frac{\sum_{t=0}^{T-1} \|\mathbf{p}_{t+1} - \mathbf{p}_t\|}{\|\mathbf{p}_{\text{Ziel}} - \mathbf{p}_0\|}. \quad (4.15)$$

Ein Wert von $\eta_{\text{Pfad}} = 1,0$ entspricht einem direkten Pfad; höhere Werte zeigen Umwege an. Die Pfادهffizienz wird nur über erfolgreiche Episoden berechnet und als Median mit IQR angegeben.

Rechenzeit pro Episode. Die Summe aller Inferenzzeiten innerhalb einer erfolgreichen Episode, gemessen auf identischer Hardware für die RL-Strategie und den Pfadplaner. Für die RL-Strategie entspricht jede Inferenz einem einzelnen Forward-Pass durch das CNN-MLP-Netzwerk; die Messung erfolgt nach GPU-Synchronisation, um asynchrone Kernelausführung nicht zu unterschätzen. Die Metrik verbindet den Rechenaufwand pro Entscheidungsschritt mit der Anzahl benötigter Schritte: Ein Verfahren, das pro Schritt schneller rechnet und zugleich weniger Schritte bis zur Landung benötigt, erzielt eine niedrigere Gesamtrechenzeit. Die Angabe erfolgt als Median mit Interquartilsabstand über alle erfolgreichen Episoden.

4.8.2 Versuchsreihe 1: Generalisierung auf unbekannte Hindernisformen

Die erste Versuchsreihe prüft, ob die trainierte Strategie auf Hindernisformen generalisiert, die während des TraiDurnings nicht gesehen wurden. Die Korridorgeometrie, die Spawnverteilung, das Landeziel und das Zeitlimit bleiben identisch zur Trainingsumgebung (vgl. Abschnitt ??). Einzig die sechs Hindernisobjekte werden durch sechs zuvor unbekannte 3D-

Objekte aus demselben Google Scanned Objects Datensatz ersetzt. Die Ersatzobjekte werden so skaliert, dass ihre horizontale Ausdehnung derjenigen der Trainingsobjekte entspricht. Die Platzierungslogik (erzwungener Slalom, alternierende Seiten) bleibt unverändert. Die Drohne muss ihre Ausweichstrategie somit rein auf Basis der Tiefenbilder auf unbekannte Geometrien übertragen.

Als Vergleichsverfahren dient ein RRT*-Pfadplaner, der auf der vollständigen Kollisionsgeometrie der Umgebung operiert: Ihm stehen die exakten Positionen und Ausdehnungen aller Hindernisse zur Verfügung. Der RL-Agent hingegen erhält ausschließlich ein lokales Tiefenbild und den propriozeptiven Zustandsvektor. Die Leistungsdifferenz zwischen beiden Verfahren quantifiziert daher den Preis eingeschränkter Sensorik, nicht den Preis der lernbasierten Methode. Beide Verfahren werden unter identischen Bedingungen evaluiert und anhand aller in Abschnitt ?? definierten Metriken verglichen.

4.8.3 Versuchsreihe 2: Transfer in eine realistische Agrarumgebung

Die zweite Versuchsreihe untersucht den Transfer der trainierten Strategie in eine photogrammetrisch erfasste Weinbergumgebung. Im Gegensatz zu Versuchsreihe 1 entfällt der Korridorbegrenzungsrahmen: Die Drohne operiert in einem offenen 3D-Scan eines realen Weinbergs, in dem die Rebreihen und das natürliche Terrain die einzigen Hindernisse bilden. Der Spawnbereich wird gegenüber der Trainingsumgebung erweitert (variablere Höhe und größerer horizontaler Versatz), und das Landeziel wird zufällig innerhalb des befahrbaren Bereichs gewählt. Die Evaluation erfolgt mit denselben Metriken und demselben Konfidenzintervallverfahren aus Abschnitt ??; als Vergleichsverfahren dient derselbe RRT*-Pfadplaner wie in Versuchsreihe 1.

5 Ergebnisse

Das sind die Ergebnisse:

Iteration	Accuracy	Precision	Recall	F1-Score	Loss
15	0.9365	0.9367	0.9365	0.9361	0.2474

Tabelle 5.1.: Ergebnisse der Testdaten

6 Diskussion

Dieses Kapitel interpretiert die in Kapitel ?? berichteten Ergebnisse im Kontext der vier Nebenfragestellungen. Jeder Abschnitt schließt mit einer expliziten Antwort auf die zugehörige Fragestellung.

6.1 Leistungsfähigkeit in der Trainingsumgebung

6.1.1 Trainingsverhalten und Konvergenz

Die drei beobachteten Phasen im Belohnungsverlauf lassen sich auf die kombinierte Dense-Sparse-Reward-Struktur zurückführen: Die dichten Komponenten (*progress*, *close*, *obstacle_proximity*) ermöglichen den initialen Anstieg, indem sie bereits für Teilfortschritte Signal liefern. Der zweite sprunghafte Anstieg nach der Stagnationsphase wird durch den sparse *success*-Reward ausgelöst, sobald der Agent konsistente Landungen erlernt. Ohne diese Kombination, also bei rein sparser Belohnung, würde dem Agenten in den frühen Trainingsphasen das Gradientensignal für Teilfortschritte fehlen, was die Konvergenz vermutlich erheblich verzögern würde.

Die im Trainingsverlauf ansteigende Landezeit und zunehmende Hindernisnähe deuten auf einen implizit gelernten Speed-Safety-Trade-off hin: Der *close*-Reward incentiviert vorsichtiges Annähern, während der Agent gleichzeitig die Proximity-Strafe zunehmend in Kauf nimmt, um Fortschritt in Richtung Ziel zu erzielen. Die Reward-Gewichtung (*progress* 1,5, *obstacle_proximity* -0,6) bestimmt den Arbeitspunkt auf diesem Spektrum.

6.1.2 Dominanter Failure-Mode und Flugverhalten

Die Failure-Mode-Analyse zeigt, dass den limitierenden Faktor darstellen.

Das qualitative Flugverhalten ist eher reaktiv als antizipierend: Der Agent nähert sich einem Hindernis auf direktem Weg, stoppt auf kurzer Distanz, weicht lateral aus und setzt den Abstieg fort. Dieses Stop-and-Dodge-Muster ist konsistent mit der lokalen Natur der Tiefensensorik, da die nach unten gerichtete Kamera Hindernisse erst bei relativer Nähe erfasst und vorausschauende Planung nur begrenzt möglich ist. Bemerkenswert ist dabei die Präzision, mit der Hindernisse in geringem Abstand passiert werden.

6.1.3 Perceptual Aliasing

Ein fundamentales Problem der gewählten Sensorarchitektur zeigt sich unmittelbar in den aufgezeichneten Tiefenbildern: Boden und Hindernisse erzeugen bei Annäherung nahezu identische Tiefensignaturen. Während die Annäherung an den Boden das Ziel darstellt, muss jene an Hindernisse vermieden werden. Der Agent löst diese Ambiguität implizit: Die relative Positionsinformation im Zustandsvektor kodiert die Entfernung zum Ziel und damit indirekt die erwartete Flughöhe, sodass das Modell lernen kann, ob eine abnehmende Tiefe Boden oder Hindernis bedeutet.

6.1.4 Beantwortung NF1

6.2 Transferleistung auf den Weinberg

6.2.1 Success-Drop und Failure-Mode-Shift

Der Leistungsabfall von Phase 1 zu Phase 2 ist erwartbar: Der Agent wurde ausschließlich in der Korridorumgebung trainiert und sieht die Weinberggeometrie erstmals zur Evaluationszeit. Aufschlussreich ist weniger der Drop selbst als dessen Zusammensetzung.

6.2.2 Einordnung als Sim-to-Sim Transfer

Die Phase-2-Evaluation ist als Zero-Shot Sim-to-Sim Transfer einzuordnen: Der Agent wird ohne Fine-Tuning und ohne Domain Randomization in einer geometrisch verschiedenartigen Umgebung evaluiert. Dass überhaupt eine partielle Übertragbarkeit besteht, lässt sich auf die umgebungsagnostische Beobachtungsstruktur zurückführen. Der Zustandsvektor kodiert relative Positionen, und das Tiefenbild enthält keine texturbasierten Merkmale, die an die Korridor-geometrie gebunden wären. Diese Einordnung sollte nicht überbewertet werden: Sim-to-Sim Transfer unter kontrollierten Bedingungen ist ein schwächeres Ergebnis als Sim-to-Real Transfer oder Generalisierung über diverse Trainingsumgebungen.

6.2.3 Perceptual Aliasing im Weinberg

Das in Abschnitt ?? beschriebene Perceptual-Aliasing-Problem bietet eine Erklärung für die erhöhte Terrain-Kollisionsrate im Weinberg. Im Korridor ist die Landefläche hindernisfrei: Sobald der Agent die letzte Hindernisschicht passiert hat, kann er das Tiefenbild in der terminalen Phase gefahrlos ignorieren und sich ausschließlich am Zustandsvektor orientieren. Dass der Agent genau diese Strategie erlernt, ist plausibel, da das Tiefenbild in Bodennähe ohnehin nur noch abnehmendes Bodenrelief zeigt und keinen handlungsrelevanten Infor-

mationsgewinn bietet. Im Weinberg bricht diese Strategie: Reihenreihen ragen als vertikale Strukturen direkt in die Landezone, sodass die Tiefenwahrnehmung gerade in der terminalen Phase entscheidungsrelevant bleibt. Ein Agent, der gelernt hat, das Tiefenbild in Zielnähe zu ignorieren, kollidiert mit Vegetation, die er bei aktiver Auswertung hätte umgehen können.

Ein möglicher Lösungsansatz wäre die Ergänzung der Beobachtung um eine semantische Segmentierung, die Boden, Hindernisse und Vegetation als separate Klassen ausweist. Anstatt die Boden-Hindernis-Unterscheidung implizit aus dem Tiefenbild ableiten zu müssen, könnte der Agent direkt lernen, welche Strukturen gefahrlos überflogen werden dürfen und welche ein Ausweichmanöver erfordern.

6.2.4 Beantwortung NF2

6.3 Leistungslücke zum Pfadplaner

6.3.1 Inferenzzeit und Echtzeitfähigkeit

Ein entscheidender praktischer Vorteil des RL-Agenten liegt in der Inferenzzeit. Ein Forward-Pass durch das Netzwerk benötigt konstante Zeit unabhängig von der Umgebungskomplexität, während RRT* iterativ Pfade sampelt und Kollisionsprüfungen auswertet. Auf eingebetteter Hardware, wie sie für reale Drohnenanwendungen relevant wäre, ist der RL-Agent echtzeitfähig, während RRT* entweder ein stark begrenztes Sampling-Budget oder leistungsfähigere Hardware erfordert.

6.3.2 Landegeschwindigkeit

Die in Abschnitt ?? beschriebene Informationsasymmetrie lässt eine deutliche Leistungslücke zugunsten des RRT*-Planers erwarten. Dass der RL-Agent dennoch vergleichbare Erfolgsraten erzielt, zeigt, dass die implizite Hindernisrepräsentation im Tiefenbild für diese Aufgabe ausreichend ist. Der RL-Agent erzielt zudem kürzere Landezeiten: Er lernt direkte Trajektorien, die keine konservativen Sicherheitsabstände einhalten, wie sie ein sampling-basierter Planer typischerweise produziert.

6.3.3 Planungshorizont

RRT* plant global: Der gesamte Pfad wird vor Flugbeginn berechnet. Der RL-Agent entscheidet lokal, pro Zeitschritt, auf Basis der aktuellen Beobachtung. In dieser Arbeit sind alle Hindernisse statisch, sodass der globale Planungshorizont von RRT* ein reiner Vorteil ist. In dynamischen Umgebungen kehrt sich dieses Verhältnis um: RRT* müsste bei jeder Ände-

rung replanen, während der RL-Agent durch seine lokale Entscheidungsstruktur inhärent auf veränderte Bedingungen reagieren kann. Dieses Argument bleibt im Kontext dieser Arbeit theoretisch, ist aber für die praktische Motivation des lernbasierten Ansatzes relevant.

6.3.4 Beantwortung NF3

6.4 Beitrag der Beobachtungskomponenten

6.4.1 Ablation: State-only vs. Vollmodell

Die Differenz in der Erfolgsrate zwischen dem Vollmodell und dem State-only-Modell quantifiziert den Beitrag der visuellen Tiefenwahrnehmung. Das State-only-Modell enthält keine Hindernisinformation und fliegt unabhängig von der Hinderniskonfiguration auf direktem Weg zum Ziel. Erfolgreiche Episoden sind daher solche, in denen die zufällige Platzierung einen freien Korridor auf der direkten Abstiegslinie belässt. Das Vollmodell zeigt dagegen laterale Ausweichmanöver vor Hindernissen: Die Tiefenwahrnehmung ermöglicht nicht nur eine höhere Erfolgsrate, sondern ein qualitativ anderes Flugverhalten.

6.4.2 Implizite Repräsentation

Welche konkreten Merkmale das CNN aus dem Tiefenbild extrahiert, bleibt unklar. Eine Grad-CAM-Analyse wurde im Rahmen dieser Arbeit nicht durchgeführt. Die Transferleistung in beiden Experimenten, auf unbekannte Hindernisformen und auf eine geometrisch verschiedenartige Umgebung, legt nahe, dass das CNN geometrische Strukturen im Tiefenbild erkennt, die über die Trainingsumgebung hinaus generalisieren. Ob diese Repräsentation einer expliziten Hinderniserkennung entspricht oder lediglich Korrelationen zwischen Tiefenmuster und erfolgreicher Ausweichrichtung kodiert, lässt sich ohne Feature-Visualisierung nicht beantworten.

6.4.3 Beantwortung NF4

6.5 Limitationen

Einzelner Seed.

Alle Ergebnisse basieren auf einem einzigen Trainingslauf. Die Bootstrap-Konfidenzintervalle quantifizieren die Evaluationsvarianz über Hinderniskonfigurationen und Startpositionen, nicht die Trainingsvarianz. Ein anderer Seed könnte zu qualitativ anderem Verhalten führen.

Die berichteten Erfolgsraten sind daher als Punktschätzung eines einzelnen Trainingslaufs zu verstehen, nicht als robuste Aussage über die Methode.

Sim-to-Real Gap.

Die Simulation bietet perfekte Sensorik ohne Rauschen, keine Windeinflüsse, keine Motorverzögerung und keine Kommunikationslatenz. Ergebnisse sind nicht direkt auf reale Drohnen übertragbar. Es wurde kein Domain-Randomization-Ansatz implementiert, der die Robustheit gegenüber sensorischen Störungen erhöhen könnte.

Reward Shaping.

Die Belohnungsgewichte wurden manuell in Vorversuchen abgestimmt. Andere Gewichtungungen könnten zu qualitativ anderem Verhalten führen. Ein systematisches Hyperparameter-Tuning der Reward-Gewichte wurde nicht durchgeführt.

Feste Zielposition.

Das Landeziel ist in allen Episoden identisch. Obwohl die Beobachtung relativ kodiert ist, könnte der Agent auf positionsspezifische Muster überangepasst sein.

Statische Umgebung.

Alle Hindernisse sind statisch. Reale Weinberge weisen bewegliche Vegetation, Wind und andere dynamische Elemente auf.

Keine Feature-Visualisierung.

Die Schlussfolgerungen über die gelernten CNN-Repräsentationen sind indirekt aus Trajektorien und Transferleistung abgeleitet. Eine Grad-CAM-Analyse oder vergleichbare Methode würde direktere Evidenz liefern.

7 Fazit und Ausblick

7.1 Beantwortung der Forschungsfragen

7.2 Limitationen der Arbeit

7.3 Perspektiven für zukünftige Forschung

Anhang

Anhangsverzeichnis

Anhang A	PID-Reglerparameter	VIII
----------	-------------------------------	------

A PID-Reglerparameter

Tabelle A1.: PID-Verstärkungen des kaskadierenden Reglers.

Regelkreis	K_p	K_i	K_d
Position x/y	1,0	0	0,7
Position z	1,5	0	1,0
Geschwindigkeit x/y	16,0	0	8,0
Geschwindigkeit z	100,0	2,0	10,0
Roll / Nick	6,0	0	3,0
Gier	0,5	0	0,8

Literatur

- Amendola, José, Linga Reddy Cenkeramaddi und Ajit Jha (2024). „Drone Landing and Reinforcement Learning: State-of-Art, Challenges and Opportunities“. In: *IEEE Open Journal of Intelligent Transportation Systems* 5, S. 520–539. ISSN: 2687-7813. DOI: 10.1109/OJITS.2024.3444487. URL: <https://ieeexplore.ieee.org/document/10637701> (besucht am 18.12.2025).
- Arafat, Muhammad Yeasir, Muhammad Morshed Alam und Sangman Moh (27. Jan. 2023). „Vision-Based Navigation Techniques for Unmanned Aerial Vehicles: Review and Challenges“. In: *Drones* 7.2. ISSN: 2504-446X. DOI: 10.3390/drones7020089. URL: <https://www.mdpi.com/2504-446X/7/2/89> (besucht am 13.05.2026).
- Bartolomei, Luca, Yves Kompis, Lucas Teixeira und Margarita Chli (Okt. 2022). „Autonomous Emergency Landing for Multicopters Using Deep Reinforcement Learning“. In: *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), S. 3392–3399. DOI: 10.1109/IROS47612.2022.9981152. URL: <https://ieeexplore.ieee.org/document/9981152> (besucht am 18.12.2025).
- Basso, Bruno und John Antle (Apr. 2020). „Digital Agriculture to Design Sustainable Agricultural Systems“. In: *Nature Sustainability* 3.4, S. 254–256. ISSN: 2398-9629. DOI: 10.1038/s41893-020-0510-0. URL: <https://www.nature.com/articles/s41893-020-0510-0> (besucht am 03.05.2026).
- Calvin, Katherine u. a. (25. Juli 2023). *IPCC, 2023: Climate Change 2023: Synthesis Report. Contribution of Working Groups I, II and III to the Sixth Assessment Report of the Intergovernmental Panel on Climate Change [Core Writing Team, H. Lee and J. Romero (Eds.)]. IPCC, Geneva, Switzerland.* Intergovernmental Panel on Climate Change (IPCC). DOI: 10.59327/IPCC/AR6-9789291691647. URL: <https://www.ipcc.ch/report/ar6/syr/> (besucht am 01.04.2026).

- Climate Change Trends and Impacts on California Agriculture: A Detailed Review* | MD-PI (2026). URL: <https://www.mdpi.com/2073-4395/8/3/25#Conclusions> (besucht am 01.04.2026).
- Cobbe, Karl, Jacob Hilton, Oleg Klimov und John Schulman (2021). „Phasic Policy Gradient“. In: *Proceedings of the 38th International Conference on Machine Learning*. Bd. 139. Proceedings of Machine Learning Research. PMLR, S. 2020–2027.
- Coumans, Erwin und Yunfei Bai (2016–2021). *PyBullet, a Python Module for Physics Simulation for Games, Robotics and Machine Learning*. URL: <http://pybullet.org> (besucht am 20.05.2026).
- Downs, Laura, Anthony Francis, Nate Koenig, Brandon Kinman, Ryan Hickman, Krista Reymann, Thomas B. McHugh und Vincent Vanhoucke (25. Apr. 2022). *Google Scanned Objects: A High-Quality Dataset of 3D Scanned Household Items*. DOI: 10.48550/arXiv.2204.11918. arXiv: 2204.11918 [cs.RO]. URL: <http://arxiv.org/abs/2204.11918> (besucht am 23.05.2026). Vorveröffentlichung.
- Duckett, Tom u. a. (2. Aug. 2018). *Agricultural Robotics: The Future of Robotic Agriculture*. DOI: 10.48550/arXiv.1806.06762. arXiv: 1806.06762 [cs]. URL: <http://arxiv.org/abs/1806.06762> (besucht am 03.05.2026). Vorveröffentlichung.
- Efron, Bradley und Robert J. Tibshirani (1994). *An Introduction to the Bootstrap*. noch lesen. New York: Chapman & Hall/CRC. ISBN: 978-0-412-04231-7.
- Food and Agriculture Organization of the United Nations, Hrsg. (2017). *The Future of Food and Agriculture: Trends and Challenges*. Rome: Food and Agriculture Organization of the United Nations. 163 S. ISBN: 978-92-5-109551-5.
- Genesis Authors (Dez. 2024). *Genesis: A Generative and Universal Physics Engine for Robotics and Beyond*. URL: <https://github.com/Genesis-Embodied-AI/Genesis> (besucht am 20.05.2026).

- Guebsi, Ridha, Sonia Mami und Karem Chokmani (2024). „Drones in Precision Agriculture: A Comprehensive Review of Applications, Technologies, and Challenges“. In: *Drones* 8.11, S. 686. DOI: 10.3390/drones8110686.
- Haarnoja, Tuomas, Aurick Zhou, Pieter Abbeel und Sergey Levine (2018). „Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor“. In: *Proceedings of the 35th International Conference on Machine Learning*. Bd. 80. Proceedings of Machine Learning Research. PMLR, S. 1861–1870. URL: <https://proceedings.mlr.press/v80/haarnoja18b.html>.
- Hodge, Victoria J., Richard Hawkins und Rob Alexander (1. März 2021). „Deep Reinforcement Learning for Drone Navigation Using Sensor Data“. In: *Neural Computing and Applications* 33.6, S. 2015–2033. ISSN: 1433-3058. DOI: 10.1007/s00521-020-05097-x. URL: <https://doi.org/10.1007/s00521-020-05097-x> (besucht am 15.04.2026).
- Hornik, Kurt, Maxwell Stinchcombe und Halbert White (1989). „Multilayer Feedforward Networks Are Universal Approximators“. In: *Neural Networks* 2.5, S. 359–366. ISSN: 0893-6080. DOI: 10.1016/0893-6080(89)90020-8.
- Hultgren, Andrew, Tamma Carleton, Michael Delgado, Diana R. Gergel, Michael Greenstone, Trevor Houser, Solomon Hsiang, Amir Jina, Robert E. Kopp, Steven B. Malevich, Kelly E. McCusker, Terin Mayer, Ishan Nath, James Rising, Ashwin Rode und Jiacan Yuan (Juni 2025). „Impacts of Climate Change on Global Agriculture Accounting for Adaptation“. In: *Nature* 642.8068, S. 644–652. ISSN: 1476-4687. DOI: 10.1038/s41586-025-09085-w. URL: <https://www.nature.com/articles/s41586-025-09085-w> (besucht am 01.04.2026).
- Jarraya, Imen, Abdulrahman Al-Batati, Muhammad Bilal Kadri, Mohamed Abdelkader, Adel Ammar, Wadii Boulila und Anis Koubaa (7. Apr. 2025). „Gnss-Denied Unmanned Aerial Vehicle Navigation: Analyzing Computational Complexity, Sensor Fusion, and Localization Methodologies“. In: *Satellite Navigation* 6.1, S. 9. ISSN: 2662-1363. DOI: 10.1186/s43020-025-00162-z. URL: <https://doi.org/10.1186/s43020-025-00162-z> (besucht am 14.05.2026).

- Kanellakis, Christoforos und George Nikolakopoulos (1. Juli 2017). „Survey on Computer Vision for UAVs: Current Developments and Trends“. In: *Journal of Intelligent & Robotic Systems* 87.1, S. 141–168. ISSN: 1573-0409. DOI: 10.1007/s10846-017-0483-z. URL: <https://doi.org/10.1007/s10846-017-0483-z> (besucht am 13.05.2026).
- Kaufmann, Elia, Leonard Bauersfeld, Antonio Loquercio, Matthias Müller, Vladlen Koltun und Davide Scaramuzza (Aug. 2023). „Champion-Level Drone Racing Using Deep Reinforcement Learning“. In: *Nature* 620.7976, S. 982–987. ISSN: 1476-4687. DOI: 10.1038/s41586-023-06419-4. URL: <https://www.nature.com/articles/s41586-023-06419-4> (besucht am 18.12.2025).
- Kim, Minwoo, Jongyun Kim, Minjae Jung und Hyondong Oh (15. Juli 2022). „Towards Monocular Vision-Based Autonomous Flight through Deep Reinforcement Learning“. In: *Expert Systems with Applications* 198, S. 116742. ISSN: 0957-4174. DOI: 10.1016/j.eswa.2022.116742. URL: <https://www.sciencedirect.com/science/article/pii/S0957417422002111> (besucht am 16.05.2026).
- Koenig, Nathan und Andrew Howard (2004). „Design and Use Paradigms for Gazebo, an Open-Source Multi-Robot Simulator“. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Bd. 3, S. 2149–2154. DOI: 10.1109/IROS.2004.1389727.
- Ladosz, Pawel, Meraj Mammadov, Heejung Shin, Woojae Shin und Hyondong Oh (Mai 2024). „Autonomous Landing on a Moving Platform Using Vision-Based Deep Reinforcement Learning“. In: *IEEE Robotics and Automation Letters* 9.5, S. 4575–4582. ISSN: 2377-3766. DOI: 10.1109/LRA.2024.3379837. URL: <https://ieeexplore.ieee.org/document/10476692/> (besucht am 14.05.2026).
- Loquercio, Antonio, Elia Kaufmann, René Ranftl, Matthias Müller, Vladlen Koltun und Davide Scaramuzza (6. Okt. 2021). „Learning High-Speed Flight in the Wild“. In: *Science Robotics* 6.59, eabg5810. DOI: 10.1126/scirobotics.abg5810. URL: <https://www.science.org/doi/10.1126/scirobotics.abg5810> (besucht am 03.05.2026).

- Makoviychuk, Viktor, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa und Gavriel State (2021). „Isaac Gym: High Performance GPU-Based Physics Simulation for Robot Learning“. In: *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*. URL: https://openreview.net/forum?id=fgFBtYgJQX_.
- Mashori, Abdul Sattar, Fuzhong Li, Muhammad Aman, Wuping Zhang, Shujie Jia, Aamir Ali, Nida Jabeen, Wangli Hao und Yuqiao Yan (1. Aug. 2026). „Remote Sensing through UAVs for Precision Agriculture: Applications, Technical Foundations, Current Barriers, and Future Opportunities“. In: *Smart Agricultural Technology* 14, S. 102074. ISSN: 2772-3755. DOI: 10.1016/j.atech.2026.102074. URL: <https://www.sciencedirect.com/science/article/pii/S2772375526002959> (besucht am 03.05.2026).
- Meier, Lorenz, Dominik Honegger und Marc Pollefeys (Mai 2015). „PX4: A Node-Based Multithreaded Open Source Robotics Framework for Deeply Embedded Platforms“. In: *2015 IEEE International Conference on Robotics and Automation (ICRA)*. 2015 IEEE International Conference on Robotics and Automation (ICRA). Seattle, WA, USA: IEEE, S. 6235–6240. ISBN: 978-1-4799-6923-4. DOI: 10.1109/ICRA.2015.7140074. URL: <http://ieeexplore.ieee.org/document/7140074/> (besucht am 14.05.2026).
- Mnih, Volodymyr, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver und Koray Kavukcuoglu (16. Juni 2016). *Asynchronous Methods for Deep Reinforcement Learning*. arXiv: 1602.01783 [cs.LG]. URL: <https://arxiv.org/abs/1602.01783>.
- Mnih, Volodymyr, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg und Demis Hassabis (Feb. 2015). „Human-Level Control through Deep Reinforcement Learning“. In: *Nature* 518.7540, S. 529–533. ISSN: 1476-4687. DOI: 10.1038/nature14236. URL: <https://www.nature.com/articles/nature14236> (besucht am 03.05.2026).

Ng, Andrew Y., Daishi Harada und Stuart J. Russell (27. Juni 1999). „Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping“. In: *Proceedings of the Sixteenth International Conference on Machine Learning*. ICML '99. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., S. 278–287. ISBN: 978-1-55860-612-8.

Patrik, Aurello, Gaudi Utama, Alexander Agung Santoso Gunawan, Andry Chowanda, Jarot S. Suroso, Rizatus Shofiyanti und Widodo Budiharto (Dez. 2019). „GNSS-based Navigation Systems of Autonomous Drone for Delivering Items“. In: *Journal of Big Data* 6.1, S. 53. ISSN: 2196-1115. DOI: 10.1186/s40537-019-0214-3. URL: <https://journalofbigdata.springeropen.com/articles/10.1186/s40537-019-0214-3> (besucht am 13.05.2026).

Peter, Robinroy, Lavanya Ratnabala, Demetros Aschu, Aleksey Fedoseev und Dzmitry Tsetserukou (25. Juni 2024). *TornadoDrone: Bio-inspired DRL-based Drone Landing on 6D Platform with Wind Force Disturbances*. Version 2. DOI: 10.48550/arXiv.2406.16164. arXiv: 2406.16164 [cs]. URL: <http://arxiv.org/abs/2406.16164> (besucht am 07.03.2026). Vorveröffentlichung.

Polvara, Riccardo, Massimiliano Patacchiola, Marc Hanheide und Gerhard Neumann (25. Feb. 2020). „Sim-to-Real Quadrotor Landing via Sequential Deep Q-Networks and Domain Randomization“. In: *Robotics* 9.1. ISSN: 2218-6581. DOI: 10.3390/robotics9010008. URL: <https://www.mdpi.com/2218-6581/9/1/8> (besucht am 07.02.2026).

Precision Landing (2026). PX4 Autopilot. URL: https://docs.px4.io/main/en/advanced_features/precland (besucht am 14.05.2026).

Radoglou-Grammatikis, Panagiotis, Panagiotis Sarigiannidis, Thomas Lagkas und Ioannis Moscholios (8. Mai 2020). „A Compilation of UAV Applications for Precision Agriculture“. In: *Computer Networks* 172, S. 107148. ISSN: 1389-1286. DOI: 10.1016/j.comnet.2020.107148. URL: <https://www.sciencedirect.com/science/article/pii/S138912862030116X> (besucht am 03.05.2026).

Rudin, Nikita, David Hoeller, Philipp Reist und Marco Hutter (2022). „Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning“. In: *Proceedings*

- of the 5th Conference on Robot Learning*. Bd. 164. Proceedings of Machine Learning Research. PMLR, S. 91–100.
- Schulman, John, Sergey Levine, Philipp Moritz, Michael I. Jordan und Pieter Abbeel (2015). „Trust Region Policy Optimization“. In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. Bd. 37. Proceedings of Machine Learning Research. PMLR, S. 1889–1897. URL: <https://proceedings.mlr.press/v37/schulman15.html>.
- Schulman, John, Philipp Moritz, Sergey Levine, Michael I. Jordan und Pieter Abbeel (2016). „High-Dimensional Continuous Control Using Generalized Advantage Estimation“. In: *4th International Conference on Learning Representations (ICLR)*. URL: <https://arxiv.org/abs/1506.02438>.
- Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford und Oleg Klimov (28. Aug. 2017). *Proximal Policy Optimization Algorithms*. arXiv: 1707.06347 [cs.LG]. URL: <https://arxiv.org/abs/1707.06347>.
- Semerikov, Serhiy O., Pavlo P. Nechypurenko, Tetiana A. Vakaliuk, Iryna S. Mintii und Andrii O. Kolhatin (28. Okt. 2025). „Vision-Based Autonomous UAV Landing: A Comprehensive Review of Technologies, Techniques, and Applications“. In: *Journal of Intelligent & Robotic Systems* 111.4, S. 115. ISSN: 1573-0409. DOI: 10.1007/s10846-025-02314-4. URL: <https://doi.org/10.1007/s10846-025-02314-4> (besucht am 05.05.2026).
- Sutton, Richard S. und Andrew Barto (2020). *Reinforcement Learning: An Introduction*. Second edition. Adaptive Computation and Machine Learning. Cambridge, Massachusetts London, England: The MIT Press. 526 S. ISBN: 978-0-262-03924-6.
- Tavasci, Luca, Francesco Nex und Stefano Gandolfi (20. Sep. 2024). „Reliability of Real-Time Kinematic (RTK) Positioning for Low-Cost Drones’ Navigation across Global Navigation Satellite System (GNSS) Critical Environments“. In: *Sensors* 24.18. ISSN: 1424-8220. DOI: 10.3390/s24186096. URL: <https://www.mdpi.com/1424-8220/24/18/6096> (besucht am 14.05.2026).

- Unde, Sanket S., V. K. Kurkute, Sachin S. Chavan, Dadaso D. Mohite, Akshay A. Harale und Ayaan Chougale (16. Sep. 2025). „The Expanding Role of Multirotor UAVs in Precision Agriculture with Applications AI Integration and Future Prospects“. In: *Discover Mechanical Engineering* 4.1, S. 38. ISSN: 2731-6564. DOI: 10.1007/s44245-025-00132-4. URL: <https://doi.org/10.1007/s44245-025-00132-4> (besucht am 29.01.2026).
- Voos, Holger und Haitham Bou-Ammar (Sep. 2010). „Nonlinear Tracking and Landing Controller for Quadrotor Aerial Robots“. In: *2010 IEEE International Conference on Control Applications*. Control (MSC). Yokohama, Japan: IEEE, S. 2136–2141. ISBN: 978-1-4244-5362-7. DOI: 10.1109/CCA.2010.5611204. URL: <http://ieeexplore.ieee.org/document/5611204/> (besucht am 13.05.2026).
- Wang, Junqiao, Zhongliang Yu, Dong Zhou, Jiaqi Shi und Runran Deng (9. Dez. 2024). *Vision-Based Deep Reinforcement Learning of UAV Autonomous Navigation Using Privileged Information*. Version 1. DOI: 10.48550/arXiv.2412.06313. arXiv: 2412.06313 [cs]. URL: <http://arxiv.org/abs/2412.06313> (besucht am 03.04.2026). Vorveröffentlichung.
- World Population Prospects* (2026). URL: <https://population.un.org/wpp/> (besucht am 03.05.2026).
- Wubben, Jamie, Francisco Fabra, Carlos T. Calafate, Tomasz Krzeszowski, Johann M. Marquez-Barja, Juan-Carlos Cano und Pietro Manzoni (12. Dez. 2019). „Accurate Landing of Unmanned Aerial Vehicles Using Ground Pattern Recognition“. In: *Electronics* 8.12. ISSN: 2079-9292. DOI: 10.3390/electronics8121532. URL: <https://www.mdpi.com/2079-9292/8/12/1532> (besucht am 14.05.2026).
- Xu, Zhefan, Xinming Han, Haoyu Shen, Hanyu Jin und Kenji Shimada (24. Sep. 2024). „NavRL: Learning Safe Flight in Dynamic Environments“. In: *arXiv preprint*. DOI: 10.48550/arXiv.2409.15634. arXiv: 2409.15634 [cs.RO].
- Yang, Tao, Peiqi Li, Huiming Zhang, Jing Li und Zhi Li (15. Mai 2018). „Monocular Vision SLAM-Based UAV Autonomous Landing in Emergencies and Unknown Environments“. In: *Electronics* 7.5. ISSN: 2079-9292. DOI: 10.3390/electronics7050073.

URL: <https://www.mdpi.com/2079-9292/7/5/73> (besucht am 07.02.2026).

Erklärung

Ich versichere, dass ich die vorliegende Arbeit mit dem Thema:

„Titel“

selbständig und nur unter Verwendung der angegebenen Quellen und Hilfsmittel angefertigt habe, insbesondere sind wörtliche oder sinngemäße Zitate als solche gekennzeichnet. Mir ist bekannt, dass Zuwiderhandlung auch nachträglich zur Aberkennung des Abschlusses führen kann. Ich versichere, dass das elektronische Exemplar mit den gedruckten Exemplaren übereinstimmt.

Leipzig, den Datum

AUTORENNAME